

Vulnerability Scanner

Ryann Stock

Felix Hoang

Daniel Lopez

Jose Sanchez

Alexia Rodriguez

Maha AlJaffan

Department of Computer Science, California State University, Fullerton

CPSC 490

Professor Velasco

April 5, 2026 (Submission 1)

May 12, 2026 (Submission 2)

Table of Contents

Table of Contents.....	2
Abstract.....	4
Introduction.....	4
Project Planning.....	6
Project Plan Overview.....	6
High-Level Goals.....	6
Phase Breakdown.....	6
Phase 1 – CPSC 490:.....	6
Phase 2 – CPSC 491:.....	7
Milestones.....	7
Work Breakdown Structure.....	8
Summary.....	10
Project Metrics.....	10
Planned vs. Actual Deliverable Completion Rate (Burndown).....	11
Status Breakdown.....	12
Deliverables Per Member.....	13
Completed Points.....	14
Task Weight.....	15
Metrics Reflection and Contribution Tracking.....	15
CPSC 491 Metrics (Implementation Phase).....	16
Planned vs. Actual Overview.....	21
Requirements Engineering.....	23
Use Cases.....	23
Use Case 1: Run Vulnerability Scanner.....	23
Modeling for Use Case 1:.....	25
Use Case 2: Pull Previous Vulnerability Scans.....	27
Modeling for Use Case 2:.....	27
Use Case 3: Configure Scan Parameters.....	29
Use Case 4: Analyze Scan Results.....	32
Use Case 5: Pull CVE/NIST Data via API.....	34
Use Case 6: Export Scan Result.....	36
Use Case 7: User Authentication and Access Control.....	37
Use Case 8: Real-Time Scan Progress Monitoring.....	38
Use Case 9: Manage Vulnerability Database (Local Cache).....	40
Use Case 10: Generate Risk Assessment Summary.....	41
Use Case 11: Notification and Alert System.....	43
Use Case 12: Schedule Automated Vulnerability Scans.....	45
Use Case 13: Vulnerability Remediation Tracking.....	46

Story Board.....	48
Architecture & Design.....	49
Client-Server Architecture.....	49
Architecture Pattern Justification.....	50
Three Tier System Pattern.....	51
Publish-Subscribe Design Pattern.....	52
Repository Design Pattern.....	53
Command Query Response Segregation.....	53
MVC Design Pattern.....	53
Pipe and Filter Pattern.....	54
Strategy Pattern.....	56
] Architecture and Design - Daniel Lopez Section.....	57
UX Design.....	62
Aesthetic.....	63
User Research.....	64
Security Analyst.....	65
Students.....	65
Small Business Technical Staff.....	66
Personas.....	67
Frameworks.....	69
Information Architecture.....	70
Architecture and Design Patterns.....	70
Hub-and-Spoke Pattern.....	71
Progressive Disclosure Design.....	72
Mock Up Photo.....	72
Prototype.....	73
Scan Results.....	74
Dashboard.....	76
New Scan.....	77
Scan in Progress.....	78
Log in UI.....	79
References.....	79

Abstract

This project encompasses the design and development of a lightweight, user-friendly vulnerability scanner. From identifying basic operating systems to testing if ports are open, it automates key reconnaissance tasks for inventory and vulnerability management. Vulnerability scanning software is commonly used by threat actors, ethical hackers, and organizations across various disciplines. The scanner performs automated host discovery, service identification, and basic vulnerability detection by analyzing exposed services and comparing them against known security weaknesses. It generates structured output that highlights potential risks and provides insight into the security posture of scanned systems. This project demonstrates how automated security tools can help defenders proactively identify vulnerabilities and improve overall system security. This is necessitated by an increase in privacy regulations, which mandate organizations to think about security and the way their data is managed. A significant portion of these organizations do not have the budget for proprietary tools or have systems capable of running modern resource-hungry applications. This leaves an open space for a lightweight solution that can be deployed easily and can quickly provide an overview of an organization's security posture to a non-trained user. It is vital to have a low barrier to entry to ensure organizations that lack trained personnel can tackle security and reduce their risk levels. Today's workforce needs trained security personnel, and part of bridging the gap means allowing non-security users to manage and take the helm when it comes to their internal security.

Introduction

The world of cybersecurity continues to expand as threat actors and ransomware groups increasingly attack infrastructure and continue to evade authorities. Organizations are left to create cybersecurity programs, the biggest component of which is vulnerability management. As organizations scan for vulnerabilities, manage, mitigate, and remediate them, they are left with increasing workloads. This project presents the design and implementation of a lightweight vulnerability scanner developed to automate the inventory of devices on the network, analyzing ports and services. While there are paid solutions out there and even free ones that are industry standard, we aim to create a lightweight solution that is capable of being placed almost anywhere. Our tool is designed for IT administrators, analysts, and red teamers. Built to be a user-friendly platform for detecting network vulnerabilities.

The application can scan TCP/UDP ports, enumerate services, and cross-reference publicly available vulnerability databases, such as National Vulnerability Database (NVD), Common Vulnerabilities and Exposures (CVE), and National Institute of Standards and Technology (NIST) registries. Then, using the (CVSS) Common Vulnerability Scoring System, the application takes all this information and then presents it to the user in an easy-to-use, simple dashboard. Making vital security reports and scan history only a click away. Our scanner does not aim to replace current tools such as OpenVAS or Nuclei, but rather to work alongside these tools to fill in any possible gaps they do not cover and to be approachable to both experienced security specialists and those newer in the industry.

Organizations today encompass both on-premises and cloud environments with a wide range of applications and possible vulnerabilities. It is no longer enough to properly configure a firewall and set inbound and outbound traffic according to the company's needs. Web-facing services, third-party attacks, phishing, and advanced malware have widened the playing field for attackers. This requires internal services to also be secure, and not just external services. This has led to the rise of concepts such as no-trust networks. However, even if implemented, a single misconfiguration could lead to compromise. By scanning ports within the network and filtering them according to the most common use cases, we can quickly take inventory of a network. Additionally, by probing open ports to see if the webpages are available, we can check and potentially probe for input and vulnerabilities. With increasing detection of services and the way they work, the version could be detected and compared to the national database of vulnerabilities. Which grants the ability to have an overview of the security posture of the organization. By tying this data to dashboards, it will grant anyone an overview of the organization and what needs to be tackled.

Organizations today need versatile tool sets to manage their inventories and protect themselves against threat actors. Privacy Regulations and laws require organizations to increase their administrative and technical controls to ensure data is appropriately secured and compliant with today's regulatory environment. There is also an increasing need for organizations to manage their internal security appropriately as Agentive cybersecurity attacks increase in severity. Internal cybersecurity programs that manage cybersecurity are recommended, and the proprietary tooling for it remains out of bounds for a significant number of businesses. There is a need for open source and free tooling, especially for low-revenue businesses or nonprofits. The IT teams remain occupied, low-staffed, or not appropriately trained. This leaves a need for simple-to-use and understandable software to increase the security posture of organizations.

Project Planning

Project Plan Overview

The Vulnerability Scanner project will be divided into two phases in conjunction with the course progression of 490 and 491. The first phase will focus on application design and documentation, which includes setting initial goals, creating well-structured diagrams and documents to be used in the next phase. The second phase, 491, will focus on the implementation process of the initial design of the project created in 490. The plan will define and outline the work breakdown, milestones, timelines, and member responsibilities required to design and develop the system between the remaining time in 490 and then committing the time given during 491 to actually building the application.

In other words, CPSC 490 establishes the architectural and conceptual design of the system, where CPSC 491 will take these concepts and theories, transforming them into a functional and testable product.

High-Level Goals

- Design the architecture of a vulnerability scanner and create documentation to support its development later for CPSC 491.
- Create a vulnerability scanner into a working application using the documentation from CPSC 490 in CPSC 491.
- Collaborate with team members and assign responsibilities during both phases.
- Deliver a working system that has been tested and is reliable by the end of CPSC 491.

Phase Breakdown

Phase 1 – CPSC 490:

Phase 1 is dedicated to designing the architecture of the Vulnerability scanner and creating professional-grade documentation to be used for implementation in the second phase. During phase 1, our group will create use cases and project requirements to assist the development team during phase 2.

Some additional objectives include:

- Defining the purpose of the project and its functional and non-functional requirements.
- Create use-case diagrams.
- Design the general architecture of the application.
- Define how data from vulnerability databases will be utilized by the scanner.
- Review and finalize documentation.

Phase 2 – CPSC 491:

Phase 2 is dedicated to creating and finalizing the application into a functional vulnerability scanner that is reliable and provides what is expected from the documentation created during phase 1. The team will decide on what tools and databases they will use to assist them in developing the application

Some additional objectives include:

- Deciding which environments will be used for development.
- Implementing a core scanning engine based on the documentation provided from the first phase.
- Adapting the development schedule to the current needs.
- Developing individual components of the application.
- Testing to validate the scanner's accuracy.
- Conduct bug fixing and performance improvements.
- Delivering the final product.

Milestones

Milestone Number	Milestones	Phase
1	Project scope and requirements completed	1
2	Design of the system architecture is complete	1
3	Full documentation is delivered	1
4	Breakdown of work and roles assigned	2
5	Workplace environment decided	2
6	Core functionality implemented	2
7	Testing completed and documented	2
8	Delivery of the application	2

Timeline

The project timeline is divided across CPSC 490 (design and documentation) and CPSC 491 (implementation and testing). The goal is to ensure steady progress with clearly defined milestones and deliverables.

CPSC 490 (Design Phase)

- Weeks 1-2: Project idea selection and approval

- Weeks 3-4: Initial requirements gathering and use case development
- Weeks 5-6: Diagram creation (use-case, state, behavioral)
- Weeks 7-8: Project planning and metrics definition
- Weeks 9-10: Documentation refinement and integration
- Weeks 11-12: Requirements refinement and gap analysis
- Weeks 13-14: Architecture preparation and technical research
- Weeks 15-16: Final documentation polishing and implementation readiness

CPSC 491 (Implementation Phase)

- Weeks 1-2: Development environment setup and tool selection
- Weeks 3-4: Core scanning engine development (port scanning, service detection)
- Weeks 5-6: Vulnerability matching integration (CVE/NVD)
- Weeks 7-8: Frontend /dashboard development
- Weeks 9-10: Testing and debugging
- Weeks 11-12: Performance improvements and feature completion
- Weeks 13-16: Final submission preparation and presentation

Work Breakdown Structure

1. Project Planning & Initialization (CPSC 490)

- a. Project Scope Definition
 - i. Define system objective and goals
 - ii. Establish constraints and assumptions
- b. Team Organization
 - i. Establish communication methods and tools
- Deliverables
 - Project proposal documentation & slides

2. Requirements Engineering (CPSC 490)

- a. Use Case Development
 - i. Identify system functionalities
 - ii. Develop use cases using the Cockburn format
 - iii. Define actors, flows, and error conditions
- b. Modeling & System Design
 - i. UML Diagram Development
 - ii. Storyboard creation
- Deliverables
 - Complete use case documentation
 - UML diagrams, storyboard

3. Project Metrics & Tracking (CPSC 490 & CPSC 491)

- a. Metric Definition
 - i. Define deliverable completion rate
 - ii. Define team contribution tracking
 - iii. Define progress tracking
- b. Metric Visualization
 - i. Design charts and graph for completion rate, contribution, and progress
- Deliverables
 - Sample charts/graphs

- Metric portion of submission document
- 4. System Architecture Preparation (CPSC 490)**
 - a. Architecture Design
 - i. Define system components (scanner, database, UI...)
 - ii. Select different architecture patterns (e.g, client-server, layered, dispatcher)
 - b. Technology Research
 - i. Identify libraries, frameworks, and tools
 - ii. Evaluate feasibility
 - Deliverables
 - Architecture diagrams
 - Technology stack (e.g, databases being used, libraries needed)
- 5. Development Environment Setup (CPSC 491)**
 - a. Environment Configuration
 - i. Set up development tools and frameworks
 - ii. Initialize version control (GitHub)
 - Repository setup
 - Environment configuration
- 6. Core System Development (CPSC 491)**
 - a. Scanning Engine Development
 - i. Implementation of port scanning
 - ii. Implementation of service detection
 - b. Vulnerability Detection Integration
 - i. Integrate CVE/NVD databases
 - ii. Implementation of a match detection service for vulnerabilities
 - Deliverables
 - Functional scanning engine
 - Modeling for vulnerability detection
- 7. Team Initialization (CPSC 491)**
 - a. Team organization
 - i. Establish team roles and responsibilities
 - ii. Organize communication methods and workflow tools
 - Deliverables
 - Role assignment and responsibility document
- 8. Front End & User Interface Development (CPSC 491)**
 - a. Dashboard Design
 - i. Design to display scan results
 - ii. Provide an interactive interface for scanning
 - b. Feature Implementation
 - i. Scan configuration interface
 - ii. Report viewer interface
 - Deliverables
 - Functional dashboard
 - Prototype UI
- 9. Testing & Debugging (CPSC 491)**
 - a. System Testing
 - i. Unit testing

- ii. Integration testing
- b. System Debugging
 - i. Identify and fix system errors
 - ii. Stabilize system performance
- Deliverables
 - Test reports
 - Debugging logs

10. Performance Optimization & Evaluation (CPSC 491)

- a. Performance Improvements
 - i. Optimize scan speed and performance
 - ii. Reduce the false positive rate
- b. Evaluation
 - i. Validate the system through test cases
 - ii. Compare planned vs actual performance
- Deliverables
 - Performance reports
 - Evaluation analysis reports

11. Final Delivery & Presentation

- a. Final System Preparation
 - i. Finalize performance
 - ii. Ensure system stability
- b. Presentation & Documentation
 - i. Prepare demo and presentation
 - ii. Gather final documentation
- Deliverables
 - Final system
 - Project deliverables (e.g, documentation, system code)

Summary

Project Metrics

The purpose of defining project metrics is to establish measurable indicators that allow us as a team to monitor our progress, identify potential problems, and ensure the final product meets the requirements defined in the initial planning phase. Metrics will be tracked across both CPSC 490 and CPSC 491 to cover documentation milestones and development progress in the 491 phase. Data will be collected weekly using GitHub commits, shared Google documents, and automated test results in 491.

CPSC 490 Metrics (Design & Documentation Phase)

1. Deliverable Completion Rate

The deliverable completion rate tracks the percentage of required documentation deliverables completed on time out of the total planned for the submission period. A healthy

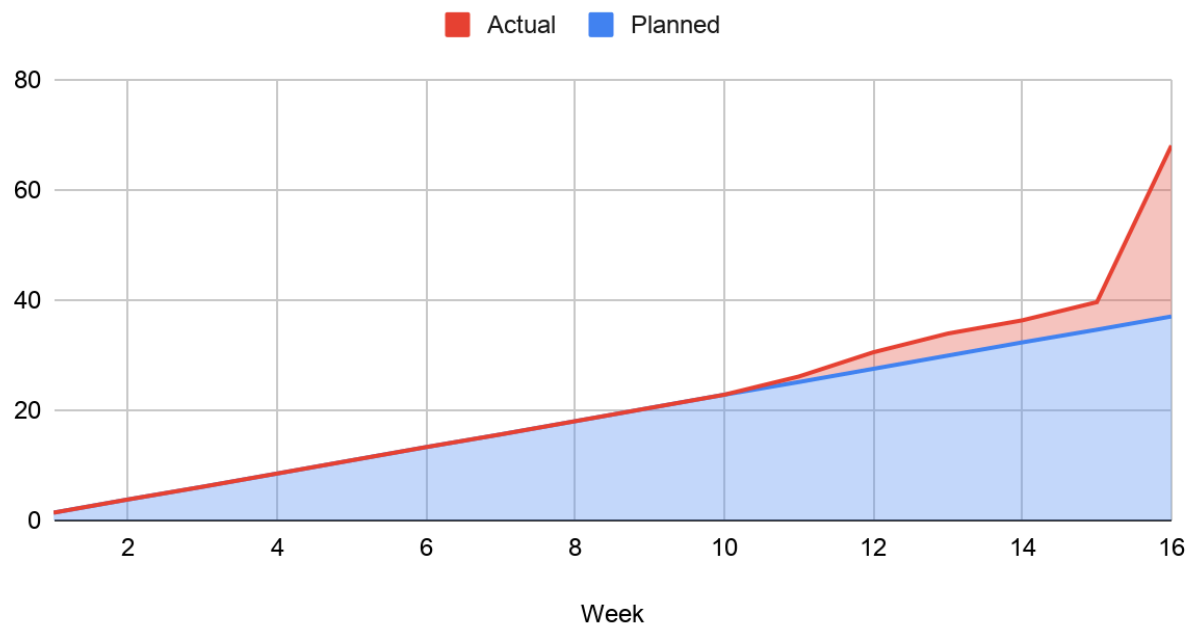
project should show a steady increase week over week, reaching its completion by the final submission deadline. This metric is tracked by maintaining a shared checklist in Google Sheets that is updated at the end of each week. If the completion rate starts to falter for more than one week, the team will hold a meeting either in person or online, talking about how we can catch up or fix our current problems to get back on track and redistribute workload.

Link to the Google sheet: [📄 Deliverable Tracker](#)

Planned vs. Actual Deliverable Completion Rate (Burndown)

Line chart comparing planning completion percentage against actual completion per week across a 16-week 490 semester. The planned line should show a steady linear increase from 0% to 100%. The actual line is updated weekly, and deviation from the planned line shows us a possibility of us falling behind.

Planned and Actual



2. Use Case & Diagram Completion

This metric tracks the number of use cases and UML diagrams completed versus the total planned for each submission. Since we have six members, each of us will be responsible for contributing to the use cases with their full modeling. The target for submission 1 is 6 use cases with at least one full modeling set, which includes a use case diagram, a state diagram, a behavioral diagram, and a storyboard.

3. Team Contribution Hours

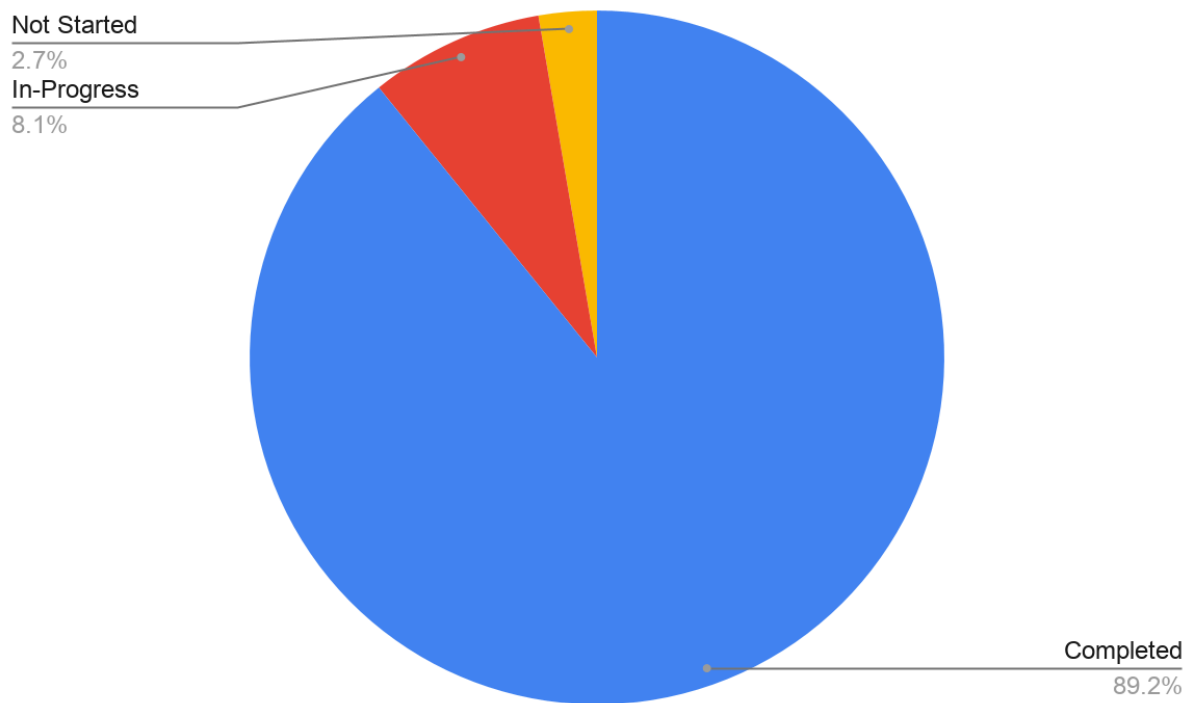
Each member logs hours per week in a shared Google sheet in addition to notifying the group chat to log in what has been contributed or changed. This metric ensures workload is distributed evenly across all members and identifies incomplete tasks so that the team understands where we are in the timeline of development. Weekly meetings and communications

would be implemented to discuss what can be done to improve our efforts and hit our desired metrics.

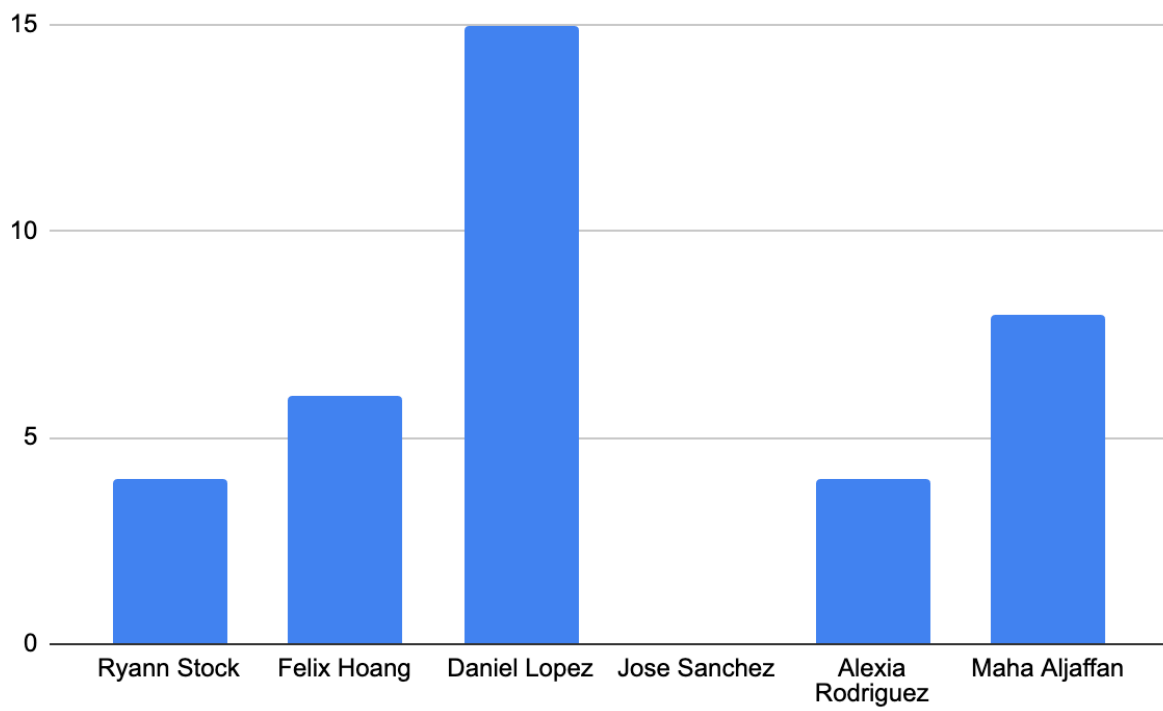
Weekly Team Contribution Hours

This is a sample chart showing how many hours we have logged in total; each bar is a person, and the bar height is the number of hours that have been contributed by everyone.

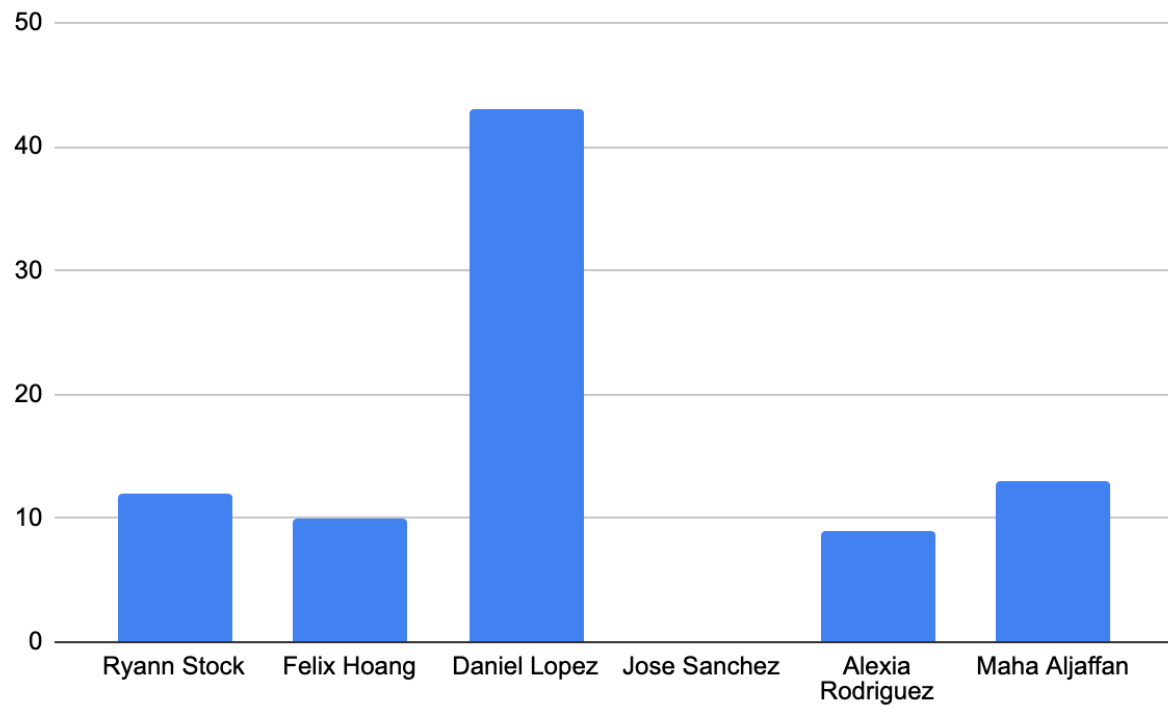
Status Breakdown



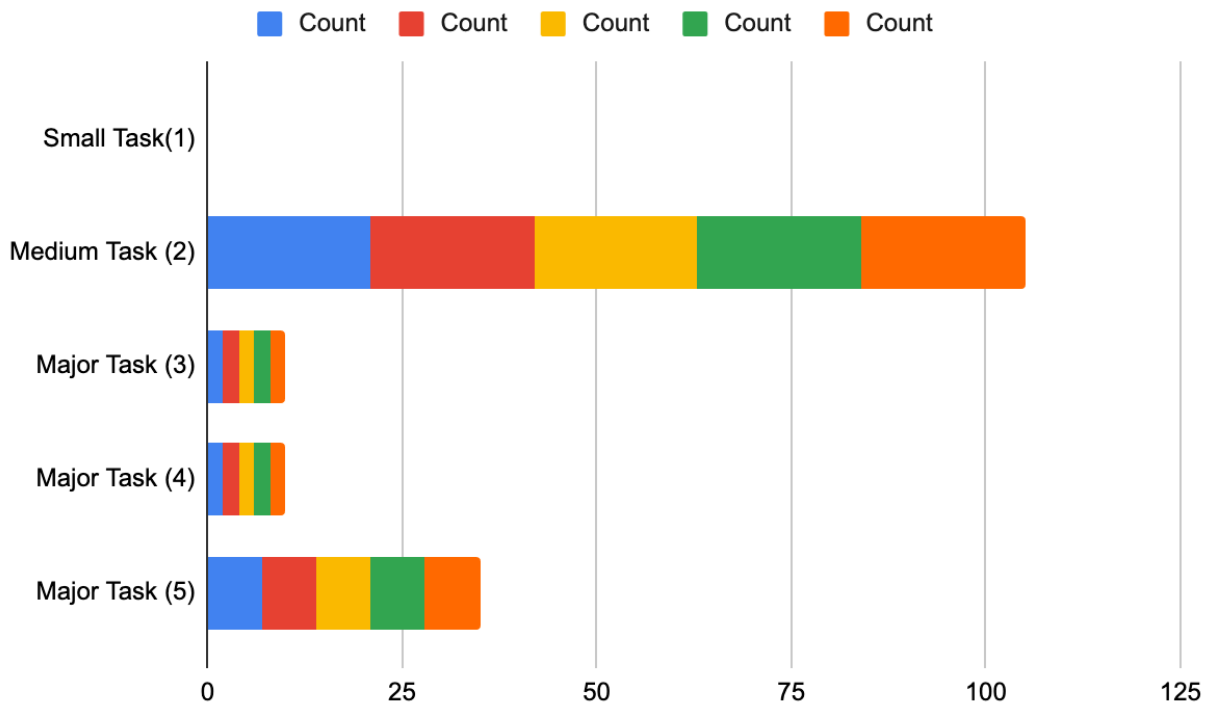
Deliverables Per Member



Completed Points



Task Weight



Metrics Reflection and Contribution Tracking

The metrics section shows its importance through ensuring the team is organized, and its ability to track progress. The most useful measurements are the ones that connect directly to visible work in the document. These include completed sections, diagrams added, prototype screens created, references added, and revisions made to earlier material. These are particle metrics because they can be checked against version history, shared planning documents, screenshots, and completed deliverables.

For this project, contribution trackers should not only count hours. Hours are a helpful type of data to keep track of but they do not always indicate the value of completed work. A better approach is to pair hours with evidence of output in the form of tasks completed. For example, a team member who spends time creating a prototype screen, revising a use case, adding a diagram, or cleaning up citations has produced work that directly supports the final submission. This makes the metric easier to keep track of because it connects team efforts to finished project requirements.

This project also uses milestone completion as a project health metric. A milestone is considered complete only when the section has written content, supporting visuals, and enough explanation to connect the work back to the vulnerability scanner. This matters because several sections require both text and diagrams. A section that only contains a heading or a genetic diagram should

be counted as fully complete.

To further support accountability and organization, the team maintained a shared Google Sheets contribution tracker throughout development. The tracking of contributions is for team members to delegate tasks formally and report what is in progress. Each completed task was logged with a description of the work performed, the team member responsible, and the importance of the task to the overall project (self reported). Tasks were categorized based on their impact on the final deliverable, such as core architecture writing, UX documentation, diagram creation, research collections, prototype development, or revision work. This allowed the team to monitor not only how much work was being completed, but also how critical each contribution was to project progress. Ideally we would want everyone to focus their efforts on completing the critical tasks as a group and take on the small to medium tasks as they come up individually.

CPSC 491 Metrics (Implementation Phase)

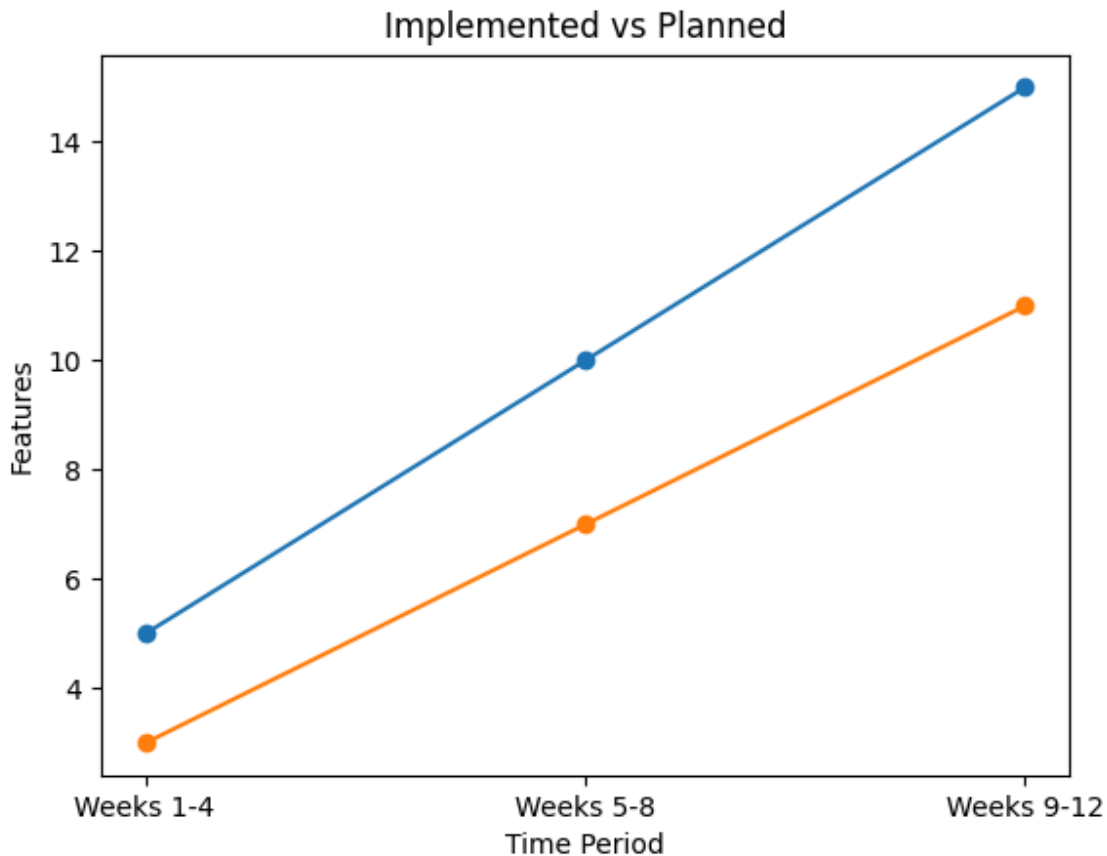
491 will be different from 490 in the sense that we shift focus from documentation to actual implementation and development. Due to this, the metrics will be more technical and have code rather than the metrics from the last phase. These metrics include feature implementation progress, code quality, and scanner performance as the scanner is built and tested.

1. Features Implemented vs. Planned

This is the primary schedule health indicator for the 491 phase. It tracks how many features were derived from the use cases defined in the documentation phase to check if it has been fully implemented versus the total number planned. Features are tracked as GitHub issues on the team's project board, with each issue corresponding to a specific use case or subcomponent. A feature is considered complete when its associated GitHub issue is closed, and implementation has been reviewed by at least 2 total team members.

In addition to tracking implementation progress, this metric also helps evaluate how accurately the original 490 planning phase translated into achievable development goals during implementation. Monitoring completed features against planned functionality provides visibility into development pacing, task prioritization, and the team's ability to adapt project scope when technical limitations or integration challenges arise during active development.

Feature Burndown Chart -



2. Bug Count: Found vs. Resolved

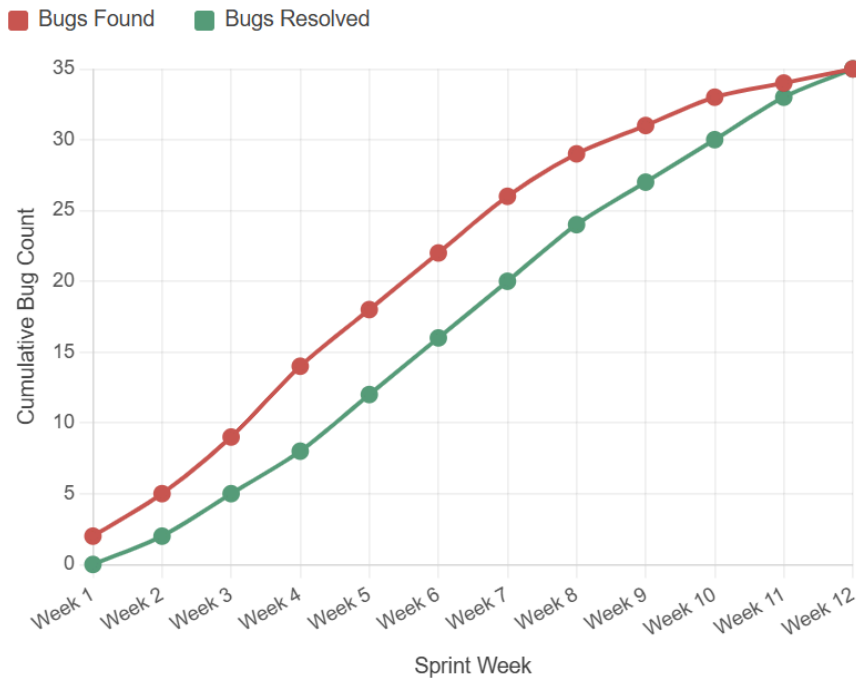
This metric tracks the number of bugs discovered during testing each week against the number of bugs resolved. Bugs are logged as GitHub issues with a bug label. Ideally, the resolved line catches up to or exceeds the found line by the end of each sprint. A large and growing gap between the two lines signals that testing is outpacing the team's capacity to fix issues and may signal a need to pause feature development and instead address the bugs.

This metric is important because it reflects both software stability and the effectiveness of the team's debugging workflow throughout the implementation phase. Tracking bug discovery against bug resolution allows the team to identify periods where system complexity increases faster than maintenance capacity. Maintaining a balanced ratio between discovered and resolved bugs helps ensure that development progress does not come at the cost of long-term reliability or scanner performance.

Bugs vs. Resolved - dual line chart

Bug Count: Found vs. Resolved

Cumulative bugs found and resolved per sprint week during CPSC 491 implementation phase



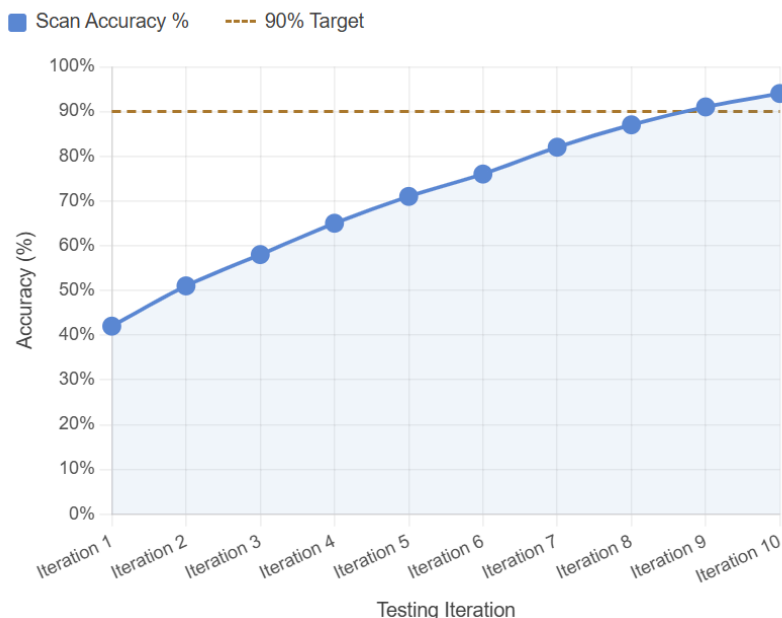
3. Scan Accuracy Rate

Specific to the vulnerability scanner, this metric tracks the percentage of known vulnerabilities correctly detected in a controlled test environment across testing iterations. A test environment with a predefined set of known vulnerabilities will be set up so that scanner accuracy can be measured objectively. As the scanner is refined through the development process, this percentage should trend upward toward a target of 90% or higher by final delivery. A stagnant or declining rate signals issues with the CVE matching logic or service detection engine. We want to use this data to measure whether the direction of our features and planning are positively contributing to the scanner's performance as development continues.

Scan Accuracy Rate Over Time –

Scan Accuracy Rate Over Testing Iterations

Percentage of known vulnerabilities correctly detected in a controlled test environment — target: 90%



4. False Positive Rate

The false positive rate tracks how often the scanner incorrectly flags a service or port as vulnerable when it's not. This metric is critical for a vulnerability scanner because a high false positive rate slowly destroys user trust in the scanner. Having a significant value in the false positive rate would indicate to us that the scanner logic is seriously wrong and needs to be looked at. The target is to reduce the false positive rate to well below 10% by the time we're close to deploying the software. This metric is measured during the same controlled test environment sessions used for scan accuracy testing.

False Positive Rate Over Time -

False Positive Rate Over Testing Iterations

Percentage of incorrectly flagged non-vulnerabilities across controlled test environment iterations
— target: below 10%

■ False Positive Rate % - - - 10% Target Threshold

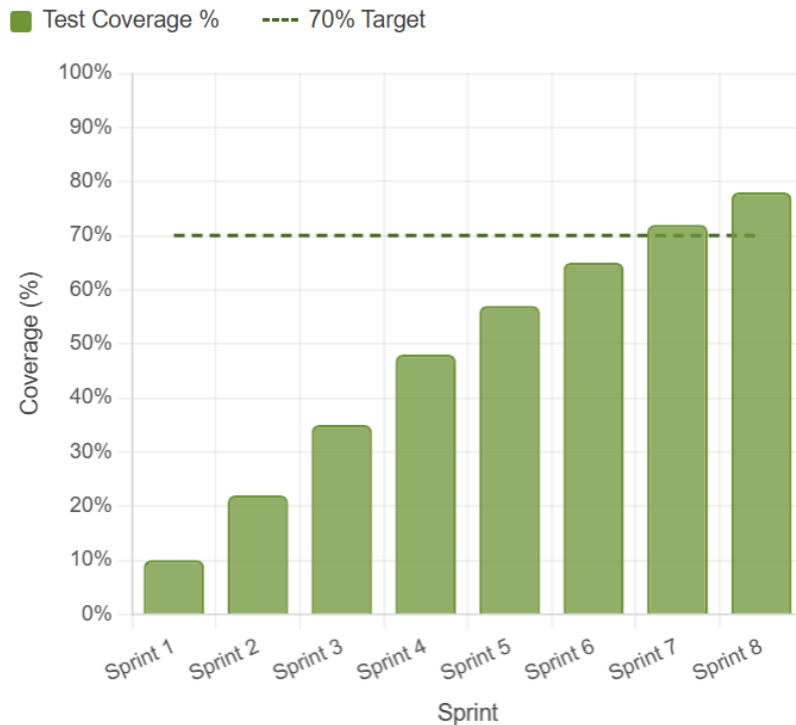


5. Test Coverage Percentage

Test coverage tracks what percentage of the codebase is covered by automated unit and integration tests. This ensures the team is not only building features but validating them systematically. Coverage reports will be generated automatically using the testing framework selected during environment setup in the 491 phase. The target coverage is 70% or higher by final delivery. In addition to a high coverage, we also want to make sure that we are exploring the different branches and conditions so that we can verify that our logic paths are performing the way we expect them to.

Test Coverage Percentage by Sprint

Percentage of codebase covered by automated unit and integration tests across CPSC 491 sprints — target: 70%



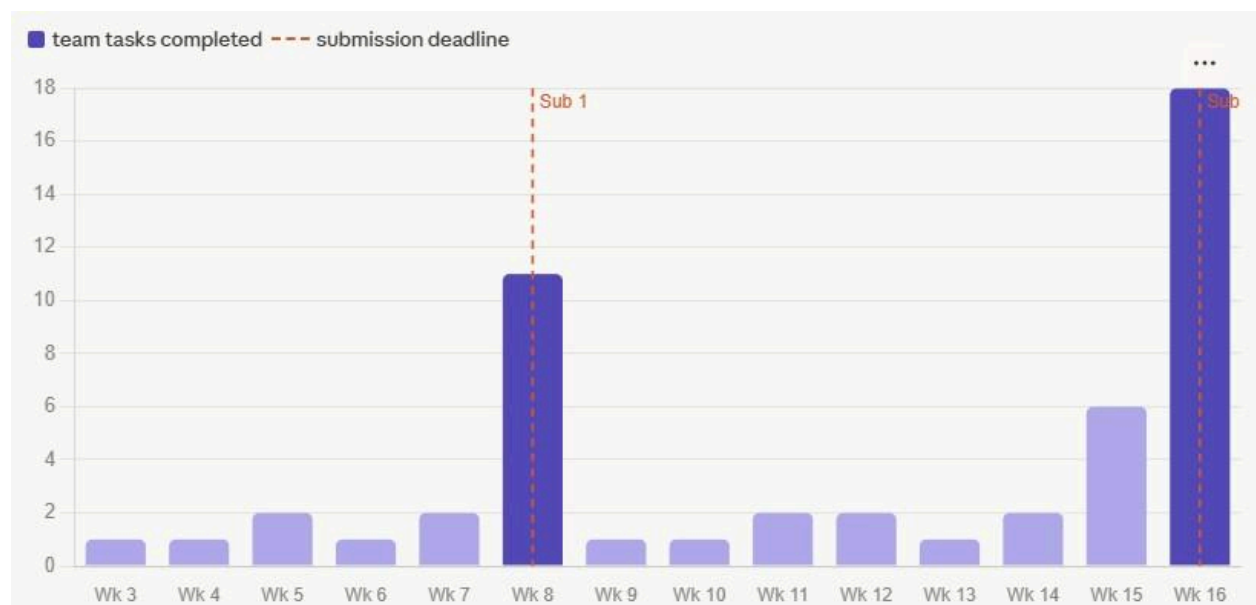
Planned vs. Actual Overview

A key component of project health monitoring is comparing the original planned schedule against actual progress at regular intervals. The Gantt chart in the Project Planning section illustrates the full planned timeline across both phases. At each milestone checkpoint, the team will record actual completion dates and compare them against the planned dates. Any milestone that slips by more than one week triggers a team review to assess the impact on downstream tasks and adjust the schedule accordingly.

Metric	Tool/Method	Responsible
Deliverable Completion Rate	Shared Google Sheet Check list	All Members
Use case and diagram completion	Google Docs submission tracker	All Members
Team contribution hours	Individual time log in Google Sheet	All Members

Features implemented vs. Planned	GitHub Projects board	All Members
Bug Count - found vs. resolved	GitHub Issues	All Members
Scan Accuracy Rate	Manual test runs in a controlled environment	Testing Leads
False Positive Rate	Manual test runs in a controlled environment	Testing Leads
Test Coverage Percentages	Automated coverage report from the test framework	Dev Lead

The productivity of all team members within Phase 1 increased significantly between weeks 15 and 16. All team members were able to contribute to their portions and complete what was needed from them to complete the CPSC 490 phase of the project. Each group member had the responsibility to contribute towards the metrics section, the architecture and design, UX design, references, and contributions to the prototype. For the architecture and design section, all group members had to deliver an architecture pattern and how it would work with our vulnerability scanner. Diagrams must be included along to further visualize how the architectures and designs would work. Group members are also expected to deliver a 2 page contribution towards the UX design. Sections include the aesthetic choice, research on users, personas, UX frameworks, UX architecture and design, and a UX mockup. The bar graph below shows the concentration of work delivered throughout the CPSC 490 timeline.



The chart above displays the total number of tasks completed by the team each week across the CPSC 490 semester, spanning from Week 3 through Week 16. The data was collected through a

shared deliverable tracker maintained by the team in Google Sheets, where each completed task was logged with the member responsible, the week of completion, and an estimated hour count.

The distribution reflects the team's natural workflow throughout the semester. During the earlier weeks of both the Submission 1 and Submission 2 phases, task completion was light, averaging one to two tasks per week as members focused on research, planning, and early drafting. Activity increased noticeably in the weeks immediately preceding each submission deadline, which is consistent with the team's approach of refining and finalizing work closer to the due date rather than submitting incomplete drafts early.

The most significant spike occurs at Week 16, the Submission 2 deadline, where the team completed 18 tasks. This reflects the volume of new sections required for Submission 2, including Architecture and Design, UX Design, Use Case reworks, Prototype work, and References, all of which were completed and merged into the final document during this period. A similar but smaller spike is visible at Week 8, the Submission 1 deadline, where 11 tasks were completed as use cases, diagrams, project planning, and the initial metrics section were finalized.

Week 15 shows a moderate uptick of 6 tasks, indicating that some members began their Submission 2 contributions a week early, which helped distribute the workload heading into the final deadline. Overall the chart confirms that all tracked deliverables were completed within the CPSC 490 timeframe and that team contribution activity was present across both submission cycles.

This pattern also highlights a common tradeoff in student project development, where work tends to cluster around assessment deadlines rather than remaining evenly distributed. While this approach increases short-term workload pressure, it also allows the team to maintain momentum and ensure consistency in writing style and technical alignment across sections. Future iterations of the project could benefit from earlier task distribution to reduce end-loaded effort and improve iteration time for feedback.

Following feedback received on the initial submission, additional commentary was added to further contextualize the productivity trends shown through the semester timeline. While the recorded metrics will be updated, the graphical data will remain unchanged, the revised discussion provides additional interpretation regarding workload concentration, submission pacing, and the relationship between project deadlines and contribution activity.

This addition helps provide a more complete explanation between project deadlines and contribution activity.

Requirements Engineering

Use Cases

Use Case 1: Run Vulnerability Scanner

Primary Actor: Security Analyst / User

Preconditions:

- Scanner is running
- The target system is reachable

Post Conditions:

- Scan results are stored
- The vulnerability report is generated

Main Success Scenario:

1. User enters target IP
2. User starts scan by selecting the "Start Scan" button
3. The system begins the scan of open ports and currently running services
4. The system compares discovered services and current versions and compares them to the vulnerability database
5. System records any found vulnerabilities
6. The system displays a vulnerability report and results from the scan

Alternative Flow:

A1 - No Vulnerabilities found

1. The system completes the scan
2. No vulnerabilities are identified
3. The system generates a report that identifies no vulnerabilities

A2 - User Cancels Scan

1. The user starts the scan
2. The user cancels that scan before completion
3. The system stops the scanning process
4. The system notifies the user that the scan has been canceled

Error Flow:

E1 - Invalid Target

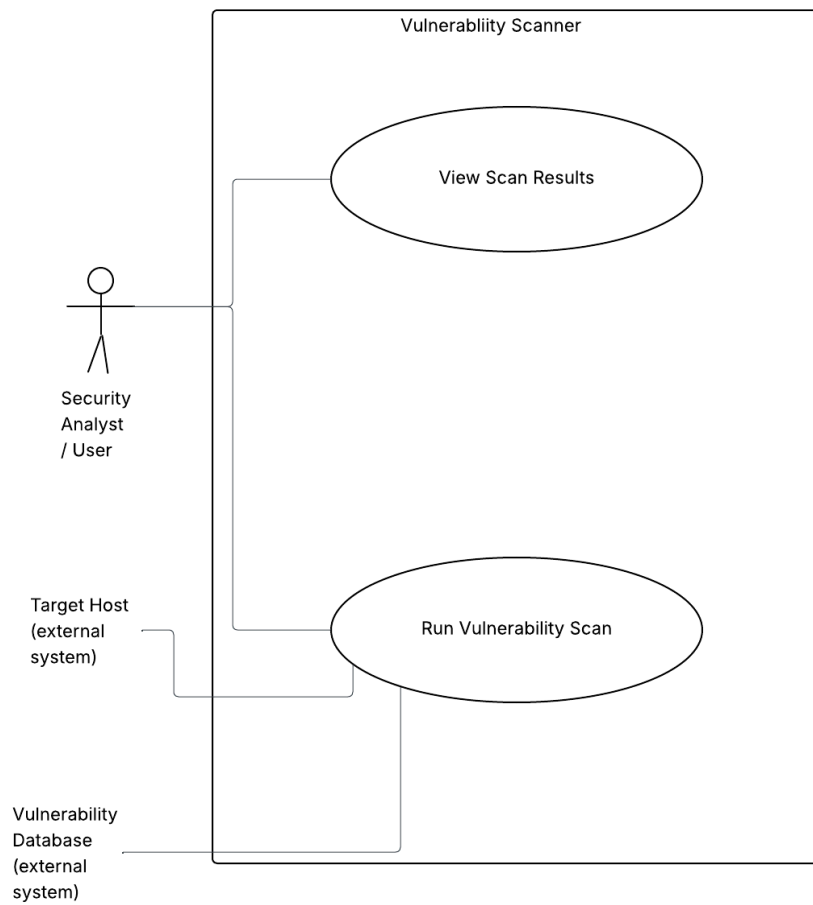
1. The user enters an invalid target ID
2. The systems verifies the target ID and detects the invalidation
3. The systems returns an error message for the invalid target ID

E2 - Vulnerability Database Unreachable

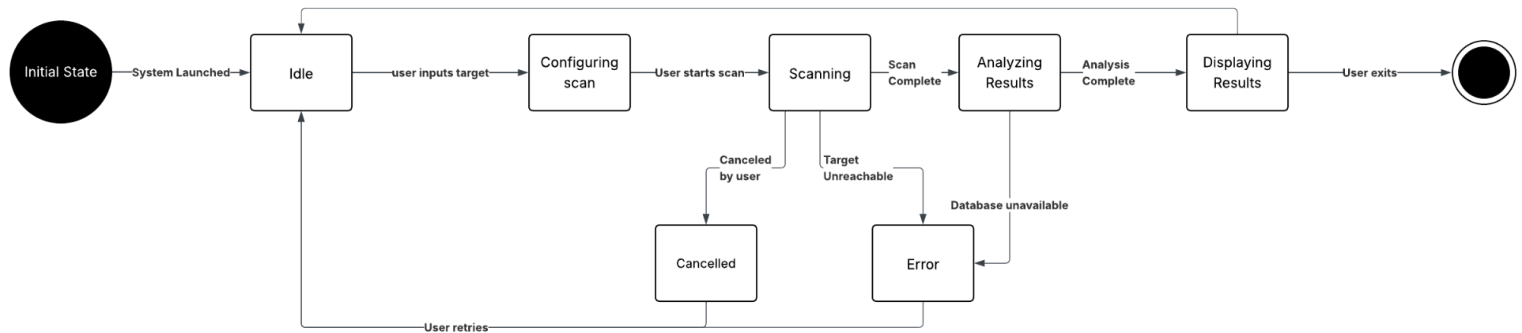
1. The system attempts to connect to the vulnerability database
2. Connection fails
3. The system logs the error and notifies the user
4. The scan stops

Modeling for Use Case 1:

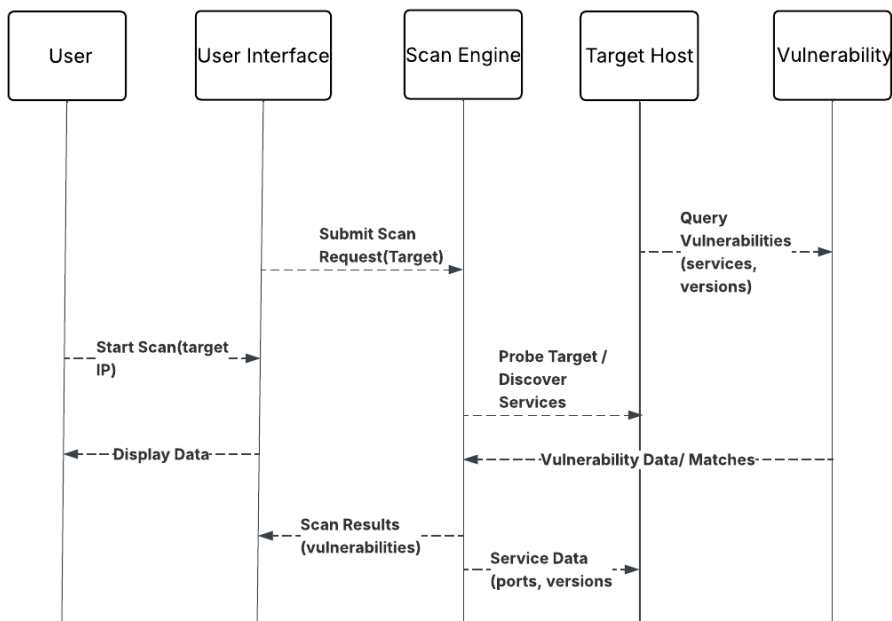
Use Case Diagram:



State Diagram:



Sequence Diagram:



Use Case 2: Pull Previous Vulnerability Scans

Primary Actor: Security Analyst / User

Preconditions:

- Scan results are already stored
- A vulnerability report has been generated

Postconditions:

- User can easily view reports and their generation date
- User can compare past reports up to 2 years

Main Success Scenario:

1. User selects Past Scans tab
2. The most recent scan is displayed
3. List of past scans is adjacent for easy access/viewing
4. The option to compare past reports is open

Alternative Flow:

A1 - No past scans found

1. Selection of the tab works
2. Empty page shows nothing except a message saying “no recorded scans/reports”

A2 - Export Record

1. Viewing the selected scan opens up the option to export as a formatted document

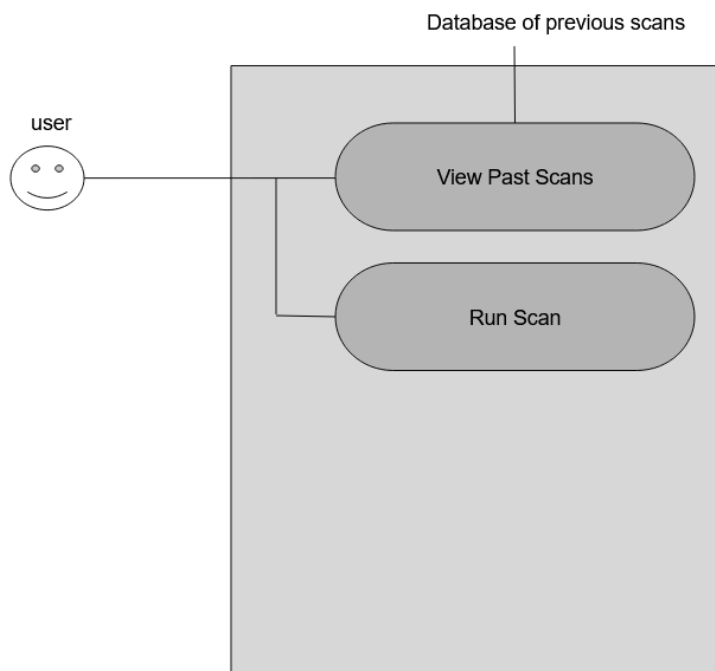
Error Flow:

E1 - Database Connection Timeout

1. User selects the tab, and the application attempts to access past records
2. Connection takes too long
3. Display retry message after first timeout

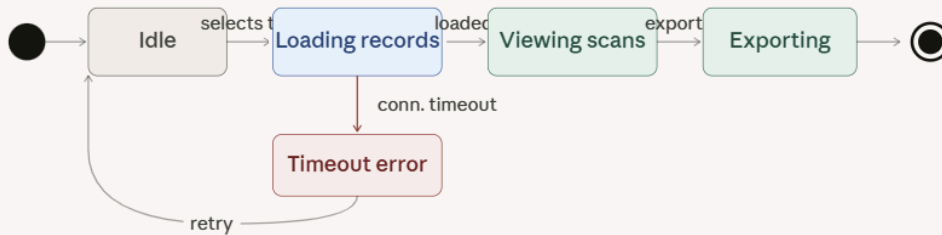
Modeling for Use Case 2:

Use case diagram:



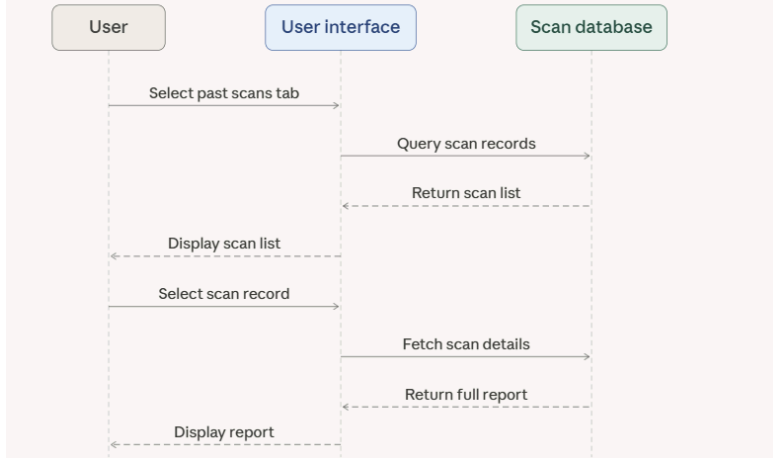
State Diagram:

Use Case 2 — State Diagram:



Sequence Diagram:

Use Case 2 — Sequence Diagram:



Use Case 3: Configure Scan Parameters

Primary Actor: Security Analyst / User

Preconditions:

- The application is running
- The user accesses the scan interface
- No scan is currently in progress

Post Conditions:

Success:

- Scan parameters are validated and saved
- System is ready to execute the scan with the selected configuration

Failure:

- Invalid parameters are rejected
- User is prompted to correct inputs

Main Success Scenario:

1. User navigates to the scan configuration interface
2. User enters the target IP address or host name
3. User specifies the port range to be scanned
4. User selects the scan depth/intensity
5. User selects the scan type
6. System validates all parameters
7. System saves scan configuration
8. System enables the option to start the scan
9. System stores configuration for reuse in future scans

Alternative Flow:

A1 - Default Parameters Used

1. User accesses the configuration interface
2. User does not modify the parameters
3. System applies default scan settings
4. System proceeds with the default configuration

A2 - Partial Configuration

1. User enters only some parameters
2. System fills missing fields with default values
3. System proceeds if the configuration is still valid

Error Flow:

E1 - Invalid Target Host

1. User enters an invalid IP address or hostname
2. System validates the input
3. System detects an invalid format or an unreachable host
4. System displays an error message
5. User is prompted to re-enter a valid target

E2 - Invalid Port Range

1. User enters a port range outside valid bounds (e.g., negative or > 65535)
2. System detects invalid port range
3. System displays an error message
4. User corrects the port

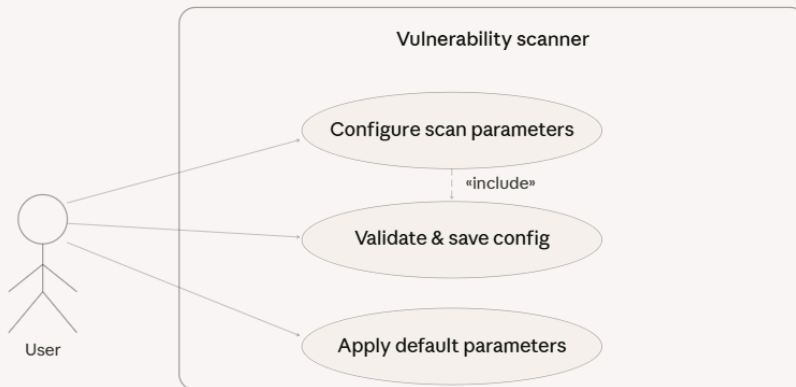
E3 - Unsupported Scan Type or Depth

1. User selects an unsupported or incompatible scan configuration
2. System detects an invalid combination
3. System displays an error message
4. User selects a valid configuration

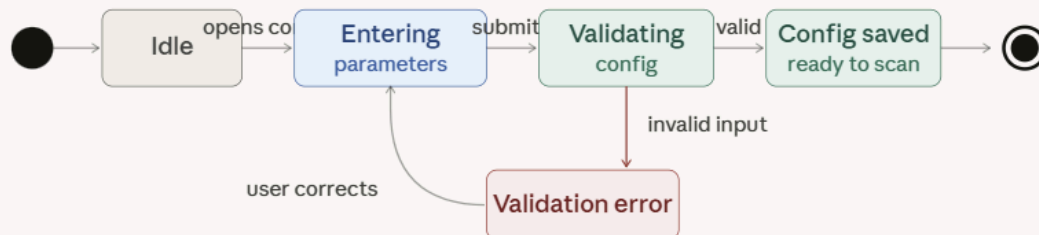
E4 - System Validation Failure

1. System attempts to validate parameters
2. Internal validation fails
3. System logs the error
4. System notifies the user and prevents the scan from starting

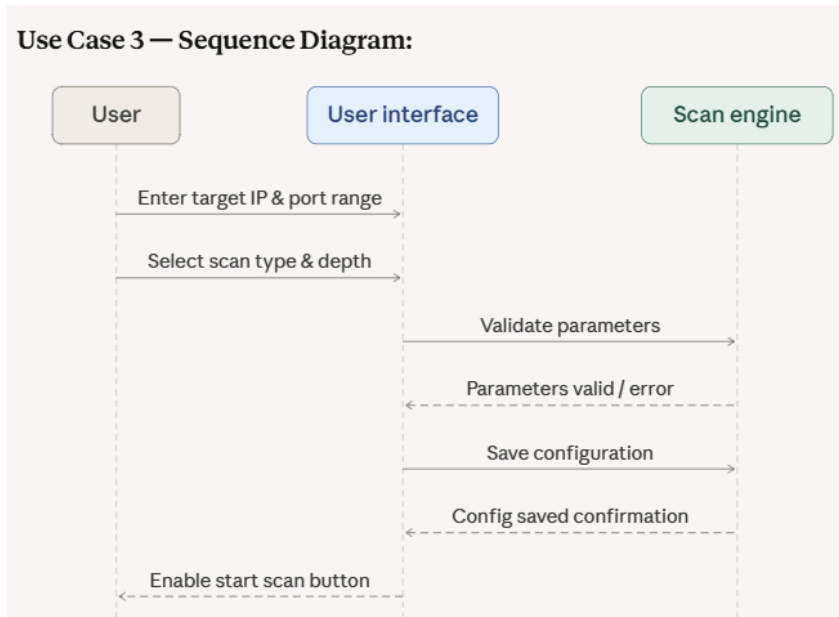
Use Case 3 — Use Case Diagram:



Use Case 3 — State Diagram:



Use Case 3 — Sequence Diagram:



Use Case 4: Analyze Scan Results

Primary Actor: Security Analyst / User

Preconditions:

- A vulnerability scan has been performed
- Scan results are available in the system

Post Conditions:

Success:

- User understands vulnerabilities and severity ratings
- System highlights critical issues
- Results are ready for further use (exporting, comparing, storing, etc)

Failure:

- Results fail to be displayed or are incomplete
- User is notified of the issue

Main Success Scenario:

1. User completes a vulnerability scan
2. System processes scan results
3. System categorizes vulnerabilities by severity (e.g., low, medium, high, critical)
4. System presents results in a structured format (dashboard/ viewable report)
5. User reviews detected vulnerabilities
6. User filters or sorts scan results (e.g., severity, host, etc.)
7. System updates the display based on the user's selection
8. User identifies vulnerabilities requiring attention

Alternative Flows:

A1 - No Vulnerabilities Found

1. Scan completes successfully
2. System analyzes results
3. No vulnerabilities are detected
4. System returns a “no vulnerabilities detected” message
5. User acknowledges results

A2 - Large Result Set

1. Scan produces numerous vulnerabilities
2. System groups or paginates results
3. User navigates through the pages of filter results

Error Flows:

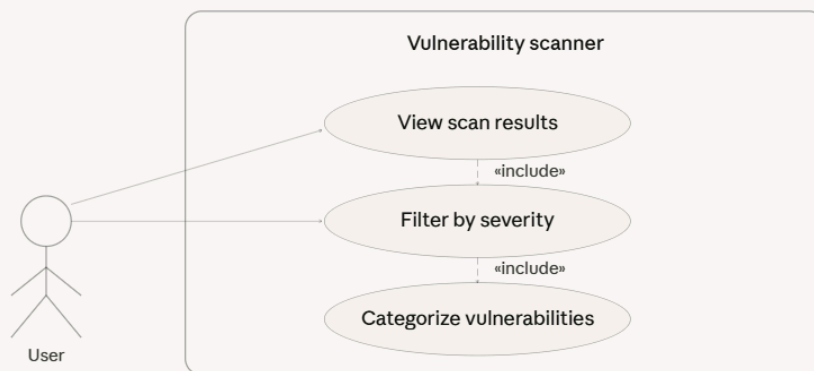
E1 - Results Processing Failure

1. Scan completes
2. System attempts to process results
3. Processing fails
4. System displays an error message
5. System logs the failure

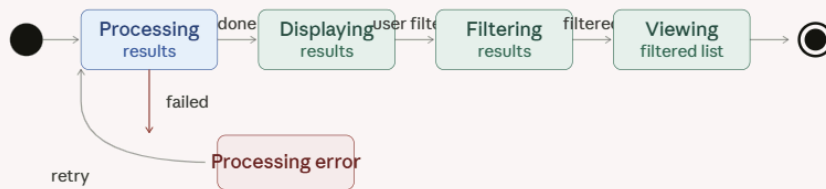
E2 - Data Retrieval Failure

1. User attempts to view results
2. System attempts to retrieve data
3. Retrieval process fails
4. System notifies the user
5. User is opted to retry

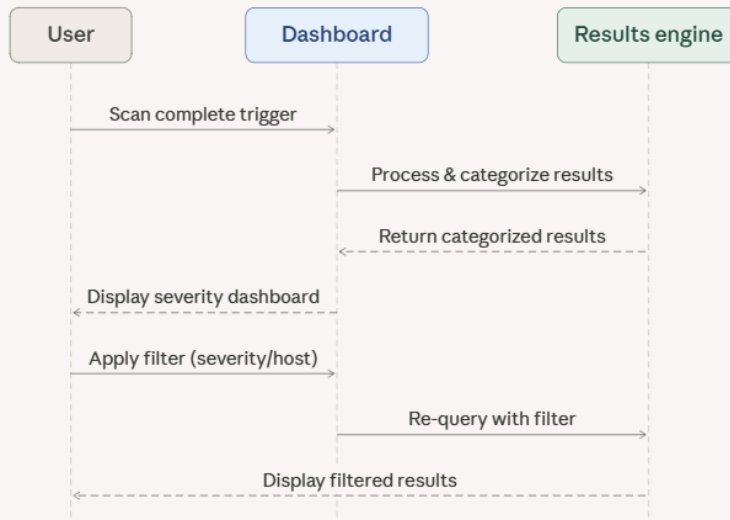
Use Case 4 — Use Case Diagram:



Use Case 4 — State Diagram:



Use Case 4 — Sequence Diagram:



Use Case 5: Pull CVE/NIST Data via API

Primary Actor: Security Analyst / User

Preconditions:

- Application has internet access
- scan engine is configured

Post Conditions:

Success:

- Internal vulnerability database is updated with the latest CVE/NIST data
- User can run scans using the most recent vulnerability information

Failure:

- Vulnerability database remains outdated
- User is notified of the issue

Main Success Scenario:

1. The system connects to CVE and NIST API endpoints.
2. The system retrieves new vulnerability entries, including CVE ID, severity (CVSS score), description, remediation steps, and NIST reference links.
3. System updates its internal vulnerability database without interruption ongoing scans.
4. User is notified when the update is complete

Alternative Flows:

A1 - API temporarily unavailable

- System logs the failure and retries after a set interval
- User can manually trigger the update later

A2 - Partial data retrieved

- System flags missing entries
- System continues updating with available data

Error Flows:

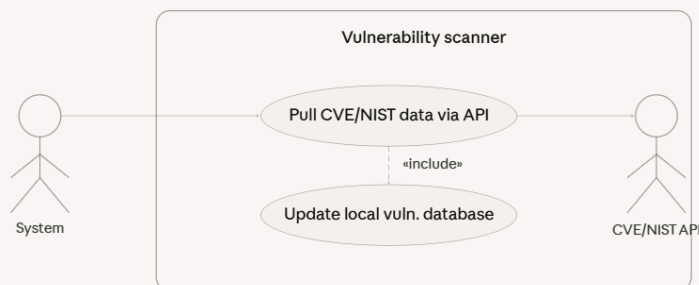
E1 - Connection Failure

- system cannot reach the API endpoints
- error is logged, user is notified, update is skipped

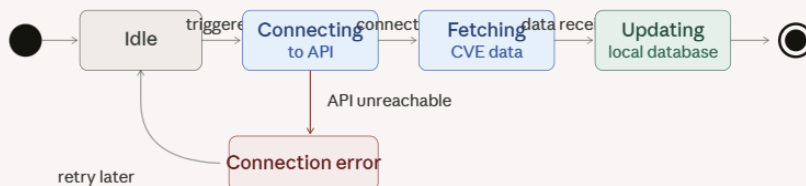
E2 - Data Validation Failure

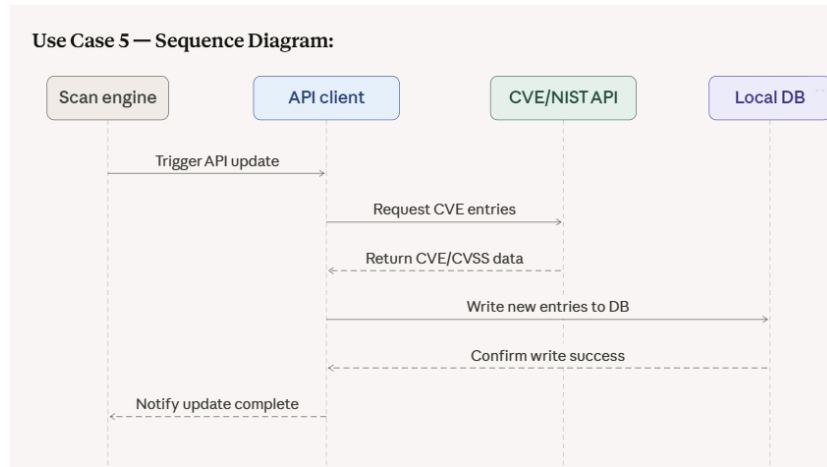
- Retrieved entries do not match the expected format
- System logs the errors and skips invalid entries
- User is notified of a partial update

Use Case 5 — Use Case Diagram:



Use Case 5 — State Diagram:





Use Case 6: Export Scan Result

Primary Actor: Security Analyst / User

Preconditions:

- Scan results are available and processed

Post Conditions:

Success:

- The report is saved in user-selected format (PDF, CSV, etc.)
- User can access the exported file externally

Failure:

- Report fails to export
- User is notified of the issue

Main Success Scenario:

1. The user selects the “Export Report” option.
2. System prompts for file format and save locations
3. User confirms export settings
4. The system generates the report, including scan summary, vulnerabilities (with CVE ID, severity, description, remediation steps, and NIST reference), and configuration details.
5. System saves the report to the chosen location
6. The user receives confirmation of successful export.

Alternative Flows:

A1 - User Cancels Export

- User aborts the process before confirming
- System cancels export and returns to the previous screen

A2 - Multiple Formats Selected

- User chooses
- System continues updating with available data

Error Flows:

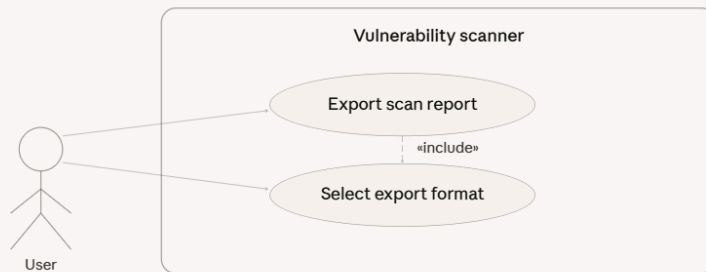
E1 - Invalid Save Location

- System cannot save to the specified location
- User is notified and prompted to choose a valid path

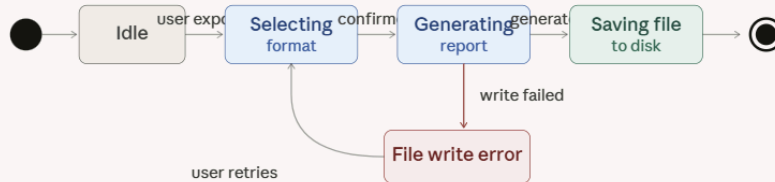
E2 -File Write Error

- system encounters a write error while saving the file
- system logs the error and alerts the user

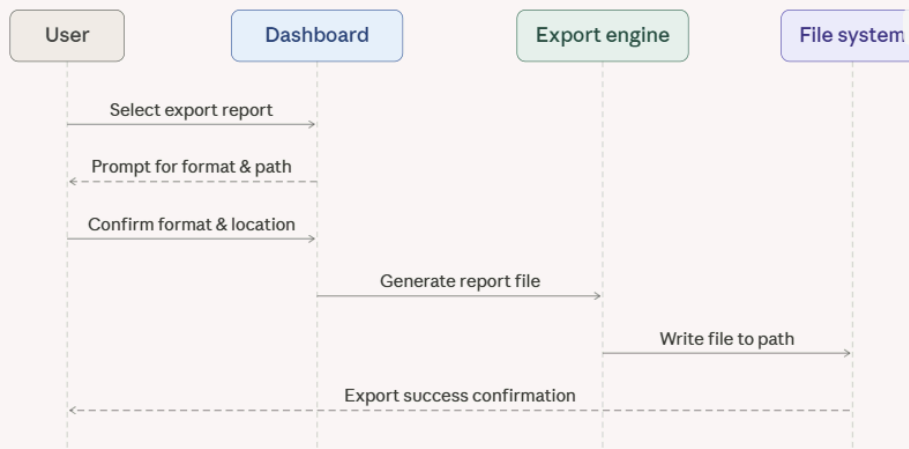
Use Case 6 — Use Case Diagram:



Use Case 6 — State Diagram:



Use Case 6 — Sequence Diagram:



Use Case 7: User Authentication and Access Control

Primary Actor: Security Analyst / User

Preconditions:

User account exists
Application is running

Post Conditions:

Success:

- User is authenticated and granted the appropriate access level
- User session is established

Failure:

- User is denied access
- System logs failed attempt

Main Success Scenario:

1. User navigates to the login interface
2. User enters credentials (username/password)
3. System validates credentials against the stored user database
4. System authenticates the user
5. System assigns role-based permissions (e.g., admin, analyst)
6. User is redirected to the dashboard

Alternative Flows:

1. A1 - Remembered Session
2. User has an active session
3. System bypasses login and grants access

A2 - Multi-Factor Authentication

1. User enters valid credentials
2. System prompts for additional verification (e.g., code)
3. User completes verification
4. System grants access

Error Flows:

E1 - Invalid Credentials

- User enters incorrect credentials
- System denies access and prompts retry

E2 - Account Locked

- Multiple failed attempts detected
- System temporarily locks the account and notifies the user

Use Case 8: Real-Time Scan Progress Monitoring

Primary Actor: Security Analyst / User

Secondary Actor: Scan Engine (system)

Preconditions:

- A scan has been successfully initiated by the user
- The scan engine is actively running and publishing status events
- The user is logged in with at least analyst-level permissions
- The dashboard is accessible and the client connection to the server is active

Postconditions:

Success:

- The user has viewed real-time progress updates including percentage completion, currently active port or service being scanned, elapsed time, and any vulnerabilities detected so far
- All progress data is accurately reflected in the dashboard without requiring a manual page refresh

Failure:

- The dashboard fails to receive live updates from the scan engine
- The system logs the communication failure with a timestamp and notifies the user that progress data may be stale
- The underlying scan continues running on the server regardless of the display failure

Main Success Scenario:

1. User initiates a scan from the scan configuration panel by entering a target IP or domain and pressing Start Scan
2. The scan engine begins execution and publishes a scan started event to the event bus
3. The dashboard subscribes to the event stream and begins polling for status updates
4. As each port is probed, the scan engine publishes incremental progress events containing the current port number, the service detected if any, and the elapsed scan time
5. The dashboard receives each event and updates the progress bar, currently scanning indicator, and elapsed time display in real time without a page reload
6. If a vulnerability is detected during scanning, the dashboard immediately flags the relevant row in the results table with the appropriate risk level color
7. Once all ports in the configured range have been scanned, the scan engine publishes a scan.complete event
8. The dashboard transitions from the live monitoring state to the completed results view and displays the full results table and risk summary bar

Alternative Flows:

A1 - User navigates away mid-scan:

1. User clicks away from the monitoring view to another section of the application
2. The scan continues running in the background on the server
3. A persistent indicator in the top navigation bar shows that a scan is in progress
4. When the user returns to the dashboard, the progress state is restored from the server and reflects the current scan status accurately

A2 - User switches to detailed log view:

1. User clicks the detailed view toggle on the dashboard
2. The system expands the monitoring panel to show a live scrolling log of raw scan output including each port probed, response time, and service banner where available
3. The user can collapse back to the summary view at any time without interrupting the scan

A3 - Scan is paused by the user:

1. User clicks the Pause Scan button during active monitoring
2. The scan engine suspends port probing and publishes a scan.paused event
3. The dashboard updates to reflect the paused state and disables the progress animation
4. The user may resume the scan, which publishes a scan.resumed event and continues from the last completed port

Error Flows:

E1 - Dashboard fails to receive progress updates:

1. System fails to update the dashboard
2. User is notified and prompted to refresh

E2 - Data Sync Delay

1. The client loses its connection to the event stream due to a network interruption
2. The dashboard detects the dropped connection after a 5-second timeout and displays a stale data warning to the user
3. The system attempts to re-establish the connection automatically up to three times
4. If reconnection succeeds, the dashboard re-synchronizes with the current scan state from the server
5. If reconnection fails after three attempts, the user is prompted to manually refresh and the scan continues on the server

E2 - Scan engine crashes mid-scan:

1. The scan engine encounters an unhandled exception and terminates unexpectedly
2. The server logs the crash with a full stack trace and timestamp

3. The event bus publishes a scan.error event
4. The dashboard receives the error event and updates the monitoring view to show the scan as failed with the last known progress state preserved
5. The user is prompted to retry the scan from the beginning or from the last completed host if partial results were saved

Use Case 9: Manage Vulnerability Database (Local Cache)

Primary Actor: Security Analyst / System

Preconditions:

- Local vulnerability database exists

Post Conditions:

Success:

- The database is updated, cleaned, or maintained
- System performance is optimized

Failure:

- The database remains outdated or corrupted

Main Success Scenario:

1. System accesses the local vulnerability database
2. System checks for outdated or duplicate entries
3. System cleans and optimizes stored data
4. System indexes vulnerabilities for faster lookup
5. System ensures compatibility with the latest scan results

Alternative Flows:

A1 - Manual Maintenance

1. User triggers database cleanup
2. The system performs the optimization process

A2 - Scheduled Maintenance

1. The system automatically performs maintenance at scheduled intervals

Error Flows:

E1 - Corrupted Data

1. System detects corruption
2. System attempts recovery or rollback

E2 - Storage Limitation

1. Database exceeds storage limits
2. System alerts the user and suggests a cleanup

Use Case 10: Generate Risk Assessment Summary

Primary Actor: Security Analyst / User

Preconditions:

- Scan results are available

Post Conditions:

Success:

- The system generates a summarized risk overview
- User understands overall security posture

Failure:

- Summary cannot be generated
- User is notified

Main Success Scenario:

1. User selects "Generate Risk Summary."
2. System aggregates vulnerability data
3. The system calculates the overall risk score based on CVSS values
4. The system categorizes risks (low, medium, high, critical)
5. The system generates a summary with key findings and recommendations
6. User reviews summarized report

Alternative Flows:

A1 - Filtered Summary

1. User selects specific hosts or a time range
2. System generates filtered risk summary

A2 - Export Summary

1. User exports summary as a standalone report

Error Flows:

E1 - Incomplete Data

1. System detects missing scan data

2. System warns the user and generates a partial summary

E2 - Calculation Error

1. System fails to compute risk score
2. System logs an error and notifies the user

Use Case 11: Notification and Alert System

Primary Actor: Security Analyst / User

Secondary Actor: Event Bus (system), Notification Service (system)

Preconditions:

- The user has configured notification preferences in the settings panel, specifying at minimum one delivery channel such as dashboard alerts, email, or both
- A scan is actively running or a scheduled scan task has been configured
- The notification service is running and subscribed to the event bus

Post Conditions:

Success:

- The user has received a timely and accurate notification for each qualifying event, specifically vulnerability detection at or above the configured severity threshold, scan completion, and scan failure
- Each notification contains enough information for the user to take immediate action without navigating to the dashboard, including the CVE ID if applicable, the affected host, the severity level, and a direct link to the full report

Failure:

- The notification service fails to deliver one or more alerts
- The system logs each failed delivery attempt with the event type, timestamp, intended recipient, and failure reason
- The underlying scan data is preserved and the user can still access the results manually through the dashboard

Main Success Scenario:

1. User opens the settings panel and configures notification preferences, selecting a severity threshold of High or above and enabling dashboard alerts
2. A scan is initiated and the scan engine begins publishing events to the event bus
3. The scan engine detects a service running Apache 2.4.49 and cross-references it against the local CVE cache, finding CVE-2021-41773 with a CVSS score of 9.8
4. The scan engine publishes a vuln.found event containing the CVE ID, severity score, affected host, affected port, and remediation summary
5. The event bus routes the vuln.found event to the notification service because the severity meets the configured threshold

6. The notification service constructs an alert containing the CVE ID, severity level, affected host and port, a plain-language description of the vulnerability, and a link to the full scan report
7. The notification is delivered to the user as a dashboard banner that persists until dismissed
8. The user clicks the notification, which navigates directly to the vulnerability detail panel for that finding
9. When the scan completes, the scan engine publishes a scan.complete event
10. The notification service delivers a scan completion summary showing total vulnerabilities found broken down by severity

Alternative Flows:

A1 - Critical Alerts Only

1. User sets notification threshold to Critical severity only in settings
2. The notification service filters all incoming vuln.found events and only processes those with a CVSS score of 9.0 or above
3. High, medium, and low severity findings are logged silently and accessible in the dashboard but generate no alert

A2 - User enables email notifications:

1. User adds an email address and enables email delivery in settings
2. The notification service delivers each qualifying alert as both a dashboard banner and a formatted email containing the same information
3. If the user is not logged into the dashboard at the time of delivery, the email serves as the primary notification channel

A3 - User activates silent mode:

1. User toggles silent mode on in settings before initiating a scan
2. The notification service continues logging all qualifying events internally but suppresses all banner and email deliveries for the duration of silent mode
3. When the user disables silent mode, a summary notification is delivered showing all events that occurred while silent mode was active

Error Flows:

E1 - Notification Service Failure

1. The event bus publishes a vuln.found or scan.complete event but the notification service is not running or is unreachable
2. The event bus queues the undelivered event for retry
3. The system retries delivery up to five times with exponential backoff
4. If all retries fail, the system logs the undelivered event with its full payload and marks it as failed in the notification log
5. The user can review undelivered notifications in the notification history panel within the settings menu

E2 - Email delivery fails:

1. The notification service attempts to send an email alert but receives a delivery failure response from the mail server
2. The system logs the failure including the recipient address, event type, and error code
3. The dashboard banner is still delivered if the user is active
4. The failed email is flagged in the notification log and the user is notified within the dashboard that email delivery encountered an issue

E2- Email Delivery Fails:

1. The notification service attempts to send an email alert but receives a delivery failure response from the mail server
2. The system logs the failure including the receipt address, event type, and error code
3. The dashboard banner is still delivered if the user is active
4. The failed email is flagged in the notification log and the user is notified within the dashboard that email delivery encountered an issue

E3 - Duplicate even received:

1. The even bus publishes the same findings event more than once due to a scan engine retry
2. The notification service detects the duplicate by comparing event IDs
3. The duplicate is discarded and only one notification is delivered to the user

Use Case 12: Schedule Automated Vulnerability Scans

Primary Actor: Security Analyst / User

Preconditions:

- User account is authenticated
- The application is running
- At least one valid scan target exists

Post Conditions:

Success:

- Scheduled scan configuration is saved
- System automatically executes scans at specified intervals

Failure:

- Scheduled task is not saved
- User is notified of configuration issue

Main Success Scenario:

1. User navigates to the scheduled scan interface

2. User selects a target host or network range
3. User selects scan frequency (daily, weekly, monthly)
4. User selects scan type and scan parameters
5. User chooses notification preferences
6. System validates scheduling configuration
7. System saves scheduled task
8. System confirms successful scheduling
9. Scheduled scans execute automatically at the configured interval

Alternative Flows:

A1 - Default Schedule Used

1. User creates a scheduled scan
2. User does not modify timing settings
3. System applies default weekly scan schedule

A2 - Multiple Scheduled Targets

1. User selects multiple hosts or ranges
2. System creates separate scheduled scan entries for each target

Error Flows:

E1 - Invalid Schedule Time

1. User enters unsupported scheduling values
2. System detects invalid configuration
3. System displays an error message
4. User corrects schedule settings

E2 - Scheduler Service Failure

1. System attempts to save scheduled task
2. Internal scheduler service fails
3. System logs the error
4. User is notified that scheduling failed

Use Case 13: Vulnerability Remediation Tracking

Primary Actor: Security Analyst / User

Preconditions:

- Vulnerability scan results exist
- Vulnerabilities have been identified within previous scans

Post Conditions:

Success:

- Vulnerabilities are assigned remediation statuses
- User can track remediation progress over time

Failure:

- Status updates fail to save
- System notifies the user of the issue

Main Success Scenario:

1. User opens vulnerability scan results
2. User selects a detected vulnerability
3. System displays vulnerability details and remediation recommendations
4. User assigns remediation status (open, in progress, resolved, accepted risk)
5. User optionally adds remediation notes
6. System saves updated remediation information
7. Dashboard updates vulnerability tracking metrics

Alternative Flows:

A1 - Bulk Status Update

1. User selects multiple vulnerabilities
2. User applies the same remediation status to all selected findings
3. System updates all selected entries simultaneously

A2 - Accepted Risk Workflow

1. User determines vulnerability cannot currently be remediated
2. User marks vulnerability as accepted risk
3. System records justification notes

Error Flows:

E1 - Database Update Failure

1. User submits remediation changes
2. System fails to save updated information
3. System logs the failure
4. User receives an error notification

E2 - Invalid Status Selection

1. User selects unsupported remediation status
2. System rejects invalid input
3. User is prompted to select a valid status

Story Board



Architecture & Design Patterns

The system needs to support users with varying levels of technical expertise, from IT administrators running scheduled scans to small business owners with no security background. It needs to remain lightweight enough to run in environments without enterprise-grade hardware. It needs to communicate with external APIs such as the NIST National Vulnerability Database in real time. And it needs to store scan history in a way that persists across sessions and can be queried efficiently.

Client-Server Architecture

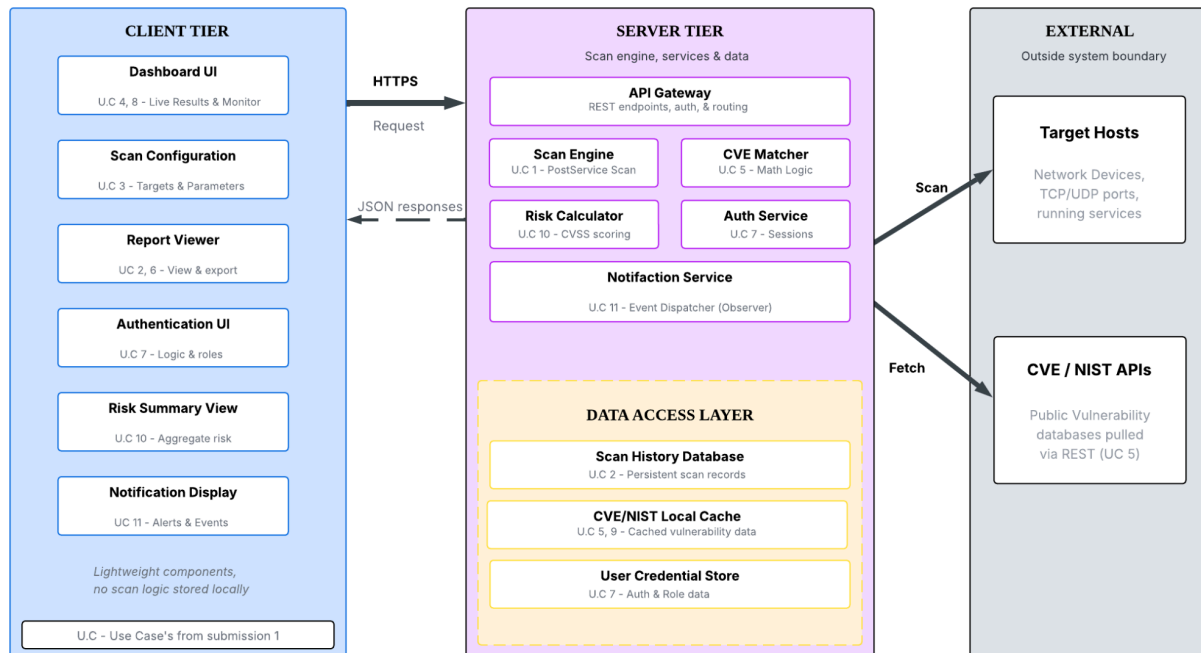
The Client-Server architecture pattern would positively impact the scanner's capabilities. In this particular architecture model, the client and servers are in communication with each other. The server distributes their service to clients who send a message out to request them, the server sends back a response once they've received the request message.

Utilizing this design structure, a central server houses the heart of the scanning core, the local CVE/NVD database, the scan history, and user authentication/notification services. Lightweight clients display the dashboard, scan configuration interface, and report interface. The goal of this section is to present why this versatile architecture pattern fits so well into the vision for our vulnerability scanner.

Two already distinguished open-source vulnerability scanners, Nessus and OpenVAS, implemented the client-server architecture into their respective designs. Josh and Singh exclusively highlight OpenVAS's client-server architecture operating through SSL communication. They also take note that host-based vulnerability assessment generally operates utilizing the client-server architecture scheme, where the client performs scans and reports the results back to a centralized server. ScienceDirect's reference on vulnerability scanning tools helps justify Nessus using a client-server architecture that includes a web-based interface and supports remote and authenticated local checks. By implementing this architecture into our product, users can have a seamless transition from the open source scanners they are already familiar with to our own.

The strongest reasoning behind choosing the client-server architecture comes from the assumptions found in our use cases. Many of the features that we want to implement in our system cannot happen cleanly within the restrictions of a single-process architecture: Take Use Case 2(Pull Previous Vulnerability Scans), which requires persistent, queryable storage of scan history that can be accessed between different sessions. This feature naturally falls onto the server-side of this proposed architecture. In Use Case 5(Pull CVE/NIST Data via API) and Use Case 9(Manage Vulnerability Database), a centralized vulnerability database is referenced where caching, cleaning, fetching, and indexing occur during an active session. The assumption of multiple role-based permissions found in Use Case 7 would require a centralized credential and session store. These are other responsibilities that fall into the server-side of this architecture. Use cases 4 & 8 describe the use of a dashboard, one for analyzing past scans and the other for real-time monitoring. This feature would be a perfect match for the design of the client-server architecture,

where clients request a certain type of information from the centralized server. A single-process type architecture would ultimately fall short in supporting these key features we plan to implement into our program design. Selecting the client-server architecture adheres to the implementation of features found in our use cases.



In Joshi and Singh's research paper, four core components for vulnerability scanners are identified: the report module, scan database, scan engine, and user interface. The client-server format fits these components into their respective groups. The scan engine, scan history database, and the CVE/NVD cache are located on the central server. Here, these components can run continuously, run continuously, and be maintained. The UI, scan configuration, and report viewer fit into the client side since they are lightweight. ScienceDirect gives an overview, noting how a modular setup such as this one grants the possibility of scalability later on without disrupting core functionality.

Architecture Pattern Justification

The selected architecture supports the main goal of the vulnerability scanner because the system depends on several separate responsibilities working together. A scan engine must discover hosts and services, a vulnerability matching component must compare service information against known vulnerability data, a database must store scan history, and a user interface must present results clearly. Keeping these responsibilities separated makes the project easier to build, test, and expand later.

A client server architecture is appropriate because the scanner is not only a local tool that runs once and disappears. The system needs persistent scan history, user authentication, role based access, report exports, and a dashboard that can display past and current results. These features are easier to manage when the main logic and data are handled by a server, while the client focuses on user interaction.

The architecture also fits the security purpose of the application. Vulnerability information from sources such as the National Vulnerability Database can be stored and updated on the server side, allowing the scanner to use a more consistent source of vulnerability data. NIST describes the NVD as a repository for vulnerability management data, including software flaws, misconfigurations, product names, and impact metrics. This supports the need for a structured vulnerability data layer inside the scanner.

The architecture also supports future expansion. In a later version, the scanner could use multiple scanning nodes that report back to one central server. This would allow the application to scan different network segments without redesigning the entire system. The current client server model leaves room for that growth while still being simple enough for the first implementation phase.

The selected architecture also improved reliability and fault isolation throughout the system. Since major responsibilities such as scan execution, vulnerability processing, database management, and user interaction are separated into distinct components, failures within one area are less likely to compromise the entire application. For example, if a client dashboard encounters an interface issue. The scan engine and vulnerability database services can continue operating independently on the server side. This separation increases system stability and simplifies troubleshooting during both development and deployment.

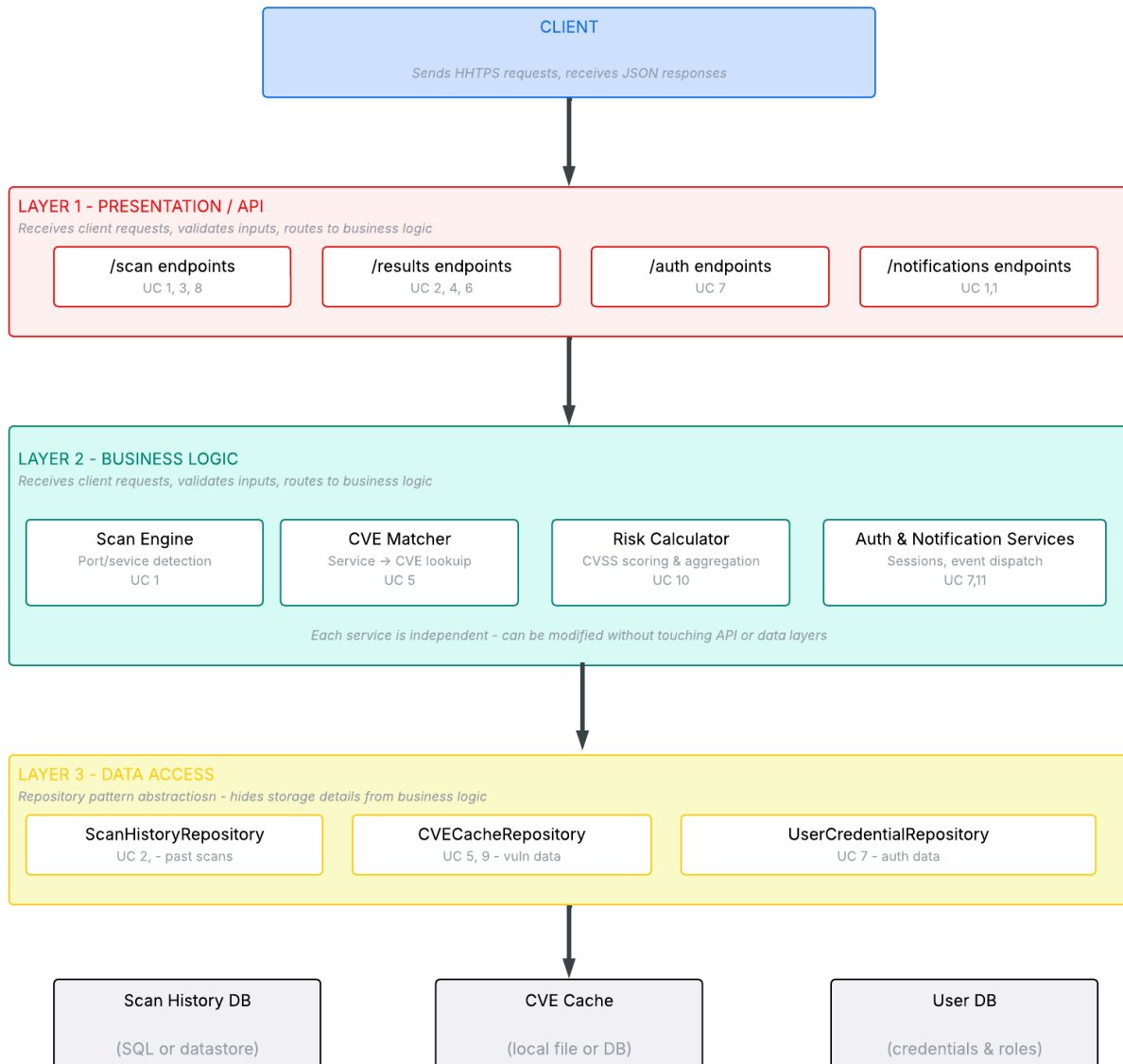
Another important advantage of this architecture is maintainability during future development cycles. As new vulnerability detection methods, reporting tools, or authentication systems are introduced. Developers can modify individual components without requiring major changes to unrelated parts of the application. This reduces development complexity and supports incremental feature expansion over time. Since cybersecurity tools must continuously adapt to evolving threats and updated vulnerability standards, maintaining a modular architecture is critical for ensuring the scanner remains scalable, adaptable, and sustainable long term.

Three Tier System Pattern

In order for the vulnerability scanner to operate in the way in which the team envisioned it, additional design patterns would need to be implemented. These design patterns would organize and support the features located in our use cases. One of these design patterns would be a layered (three-tier) pattern on our central server. To account for business use as well as individual personal use, the server would have three logical layers: an API layer, a business logic layer, and a data access layer. The API layer would handle any client requests coming into the server. The business logic layer contains the scan engine, CVE matching logic, and risk assessments. The data access layer accesses the CVE/NVD caches and our scan history databases. ScienceDirect describes the structure of this three-layered model as scalable and maintainable over the two-tier system due to the versatility in modifications. The three-tiered system can have the specific portions of each layer modified without having to completely scrap other layers. Since our goal is to keep the scanner open-source, this three-layered system allows for specific portions of the scanner to evolve without having to completely rewrite functions that have little relation to each other.

Diagram 2: Three-Tier Layered Architecture

Internal Server Organization - Layered Pattern within Client-Server style



Publish-Subscribe Design Pattern

Another design pattern that would compliment the system would be the Publish-Subscribe pattern. This feature would be geared around the events generated throughout the system. UC-8 and UC-11 contain features that grant live updates throughout the scan and notifications for the user when a vulnerability is detected or upon completion. The publish-subscribe pattern allows the system to push out events while the dashboard and notification services are subscribed to them. This creates decoupling amongst event producers and the consumers in our system.

Implementing this feature simplifies further additions of real time monitoring and other event driven features without modifying the scan engine.

This pattern also improves scalability and responsiveness within the vulnerability scanner architecture. Since multiple services can subscribe to the same event stream, new components can be integrated into the system without significantly increasing coupling between modules. For example, future implementation could include audit logging services, analytics dashboards or automated remediated systems that react to vulnerability events in real time while still operating independently from the scan engine itself.

Additionally, the Publish-Subscribed pattern improved maintainability by separating even generation from event handling responsibilities. The scan engine only needs to publish system events, while subscriber services determine how those events are processed or displayed. This reduces direct dependencies between components and makes the overall architecture easier to modify, test, and expand during future development phases in CPSC 491.

The Published-Subscribe pattern also reduces communication overhead compared to a polling-based system. Instead of clients continuously requesting updates from the server, the server publishes events only when meaningful system changes occur. This allows real-time scan monitoring to remain efficient during large scans involving numerous hosts, vulnerabilities, or active notifications.

Examples of published system events include:

- scan.started
- scan.progress
- vuln.detected
- scan.completed

Subscribers to these events may include:

- dashboard UI
- notification services
- logging systems
- report generation systems

Repository Design Pattern

In software development, the repository pattern is used in the abstraction of data retrieval/storage logic and business logic. The repository is situated between the domain and the data access. The repository's job is to access and manage the data while the domain layer provides a list of objects and application rules. The primary functions of this pattern are to optimize and organize the codebase so that the general architecture of the application is more modular. The modular nature of this pattern allows for us to scale. Repository design pattern is based in the modeling philosophies of domain driven design, which places importance upon improving the software by clearly defining boundaries between services and unifying business and technical language.

The major advantage of using this design is the fact that it allows us to place our complex data operations behind a business layer. The user interacts with the business layer to perform complex querying operations, without actually having to go through the act of going through the tedious process. The repository layer is called upon by the business layer to retrieve this information, the business layer is forwarded the data and presents it to the user.

Command Query Response Segregation

With how much data and configurations our vulnerability scanner will have to query through, it's going to call upon read operations more often than write operations. We recommend using command query response segregation (CQRS), this design pattern separates read and write operations into different models within the application. So one schema would be optimized for read operations and one can be configured for write operations and this allows us to be more intentional with our permissions and access. This approach simplifies the design of the system and addresses the differences in how frequently these operations are called for use. This creates a more flexible and overall modular system design but comes at the cost of being a challenging pattern to implement.

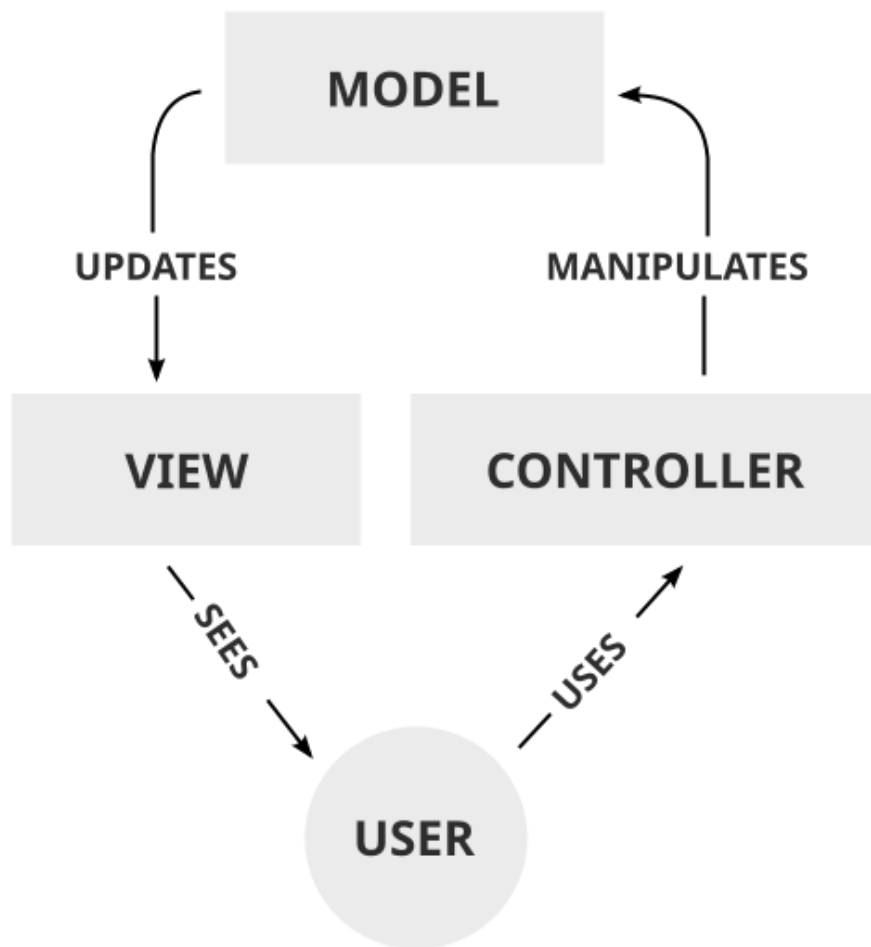
One of the major challenges is ensuring the synchronization of data being written and stored. The goal here is to have the write operations immediately update the corresponding database, and that new information to be readily accessible through the read operations. If we do not have this aspect in our design, then we risk utilizing information from outdated databases. We can squeeze out better performance thanks to the clear division between our operations

MVC Design Pattern

The final design pattern that ties the whole system structure together is the Model View Controller pattern. This design will be specifically placed on the client side of the architecture because typically it is used in GUIs. The model will assist in holding scan results and configuration state, the controller will take care of user actions such as initializing scans, applying filters or triggering exports. The addition of the MVC pattern ensures the user remains easy to scale in the long-term without making any system altering changes to the scan engine or vulnerability database.

The MVC pattern is particularly important for supporting different categories of users identified throughout the UX research and persona analysis sections of the project. Technical users such as IT administrators may prioritize detailed vulnerability data, filtering tools, and export functionality, while less technical users may require simplified dashboards and more accessible visual summaries. By separating the Mode, View, and Controller components, the interface can evolve for different user contexts without requiring changes to the backend processing logic.

The Model component stores scan configurations, vulnerability results, and application state information. The View component visually renders dashboards, charts, tables and reports for the user. The Controller processes user interactions such as initiating scans, filtering vulnerabilities, exporting reports, or navigating scan history. Separating these responsibilities improves maintainability and allows interface components to scale alongside future feature development.



Visual aid to better the flow of MVC design interactions.

The model is an important component to this pattern, it is what connects the view and the controller, essentially acting as the manager between the other two components. The view component visually renders the information and data from the model for the user's benefit. This would be in the form of visual charts or menu options for the purpose of taking user input and forwarding that information to the controller component.

The controller takes the user input from the view component and uses that information to update or manipulate the model's settings. Once the input is received, the model then uses this new data and updates the view component.

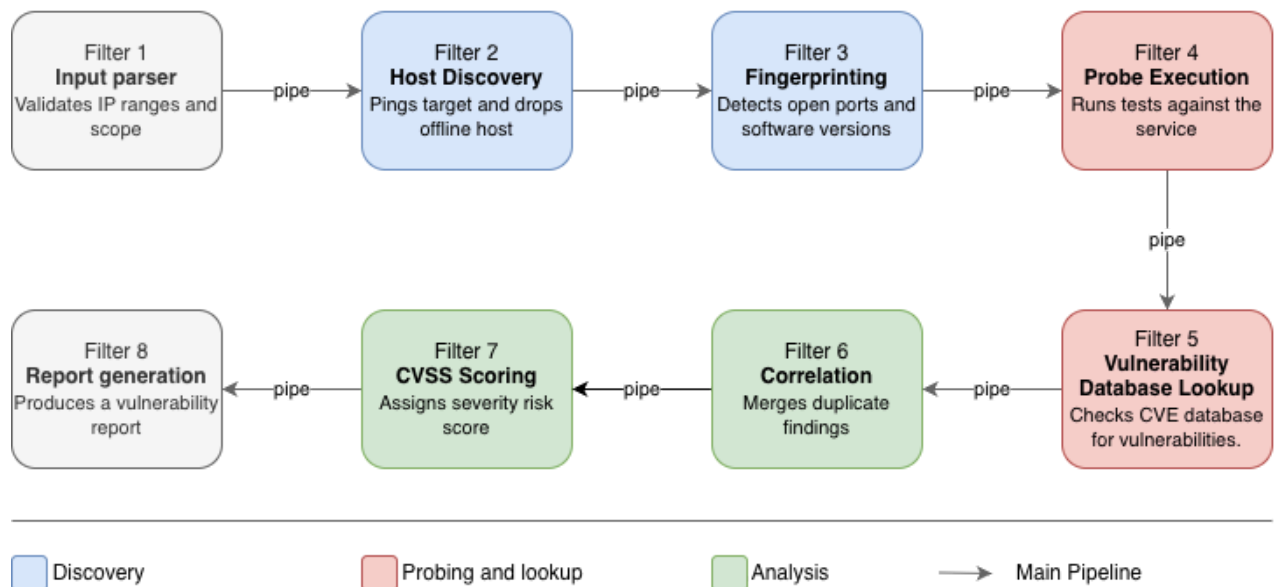
Pipe and Filter Pattern

The Pipe and Filter architecture pattern is a pattern that's able to organize a whole system into units called filters which transform the data and processes that are inputted into the program. Pipes are then used to connect each filter to the next. This pattern allows great flexibility when a program does a task (Hasan, 2019).

When it comes to applying this architecture pattern for a vulnerability scanner, data is able to flow from target to target, ensuring that the software is able to return the best response. It starts with a system to assess. The scanner looks for an active host, detects any missing patches or outdated software, tests the applications for vulnerabilities, generates a score, and finally adds all the data into a report.

The diagram below shows the filters of the vulnerability scanner. Each box is its own filter which is connected to the main pipeline. The pipeline is linear, thus making the diagram linear.

Pipe and Filter Pattern: Vulnerability Scanner



Each filter only has access to the information temporarily and forgets the data right after processing. This allows the filter to only take information from the previous pipe and give it to the next one. They are unaware of any other filters in the whole architecture. This allows for the reorganization of filters, easy updates, logic replacement, running filters on different hardware, or running multiple filters in parallel at the same time (Pipes and Filters pattern, 2026).

- Filter 1: Input Parser. Takes the input of the user, such as an IP address of domain name and checks if they are valid to take in for the next stage.
- Filter 2: Host discovery. Sends signals to the address to assess whether it is online and responding or not.
- Filter 3: Fingerprinting. Scans the host to see which ports are open and which software the system is running on.
- Filter 4: Probe Execution. Runs the vulnerability tests and collects results.
- Filter 5: Vulnerability Database Lookup. Checks an official vulnerability database to see if any of the discovered issues have an existing severity score, a classification, or a publicly available exploit already existing.
- Filter 6: Correlation. Merges duplicate findings to avoid overlaps.

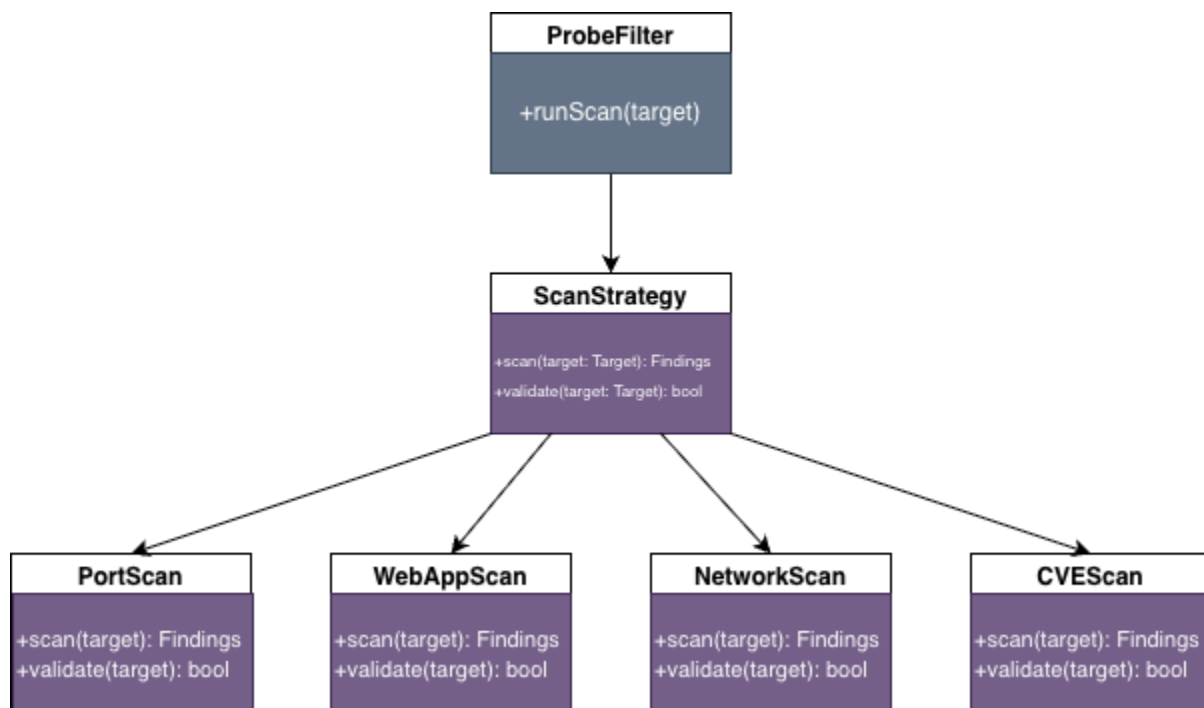
- Filter 7: CVSS Scoring. Takes the results from the vulnerability database and calculates a risk score for each issue found, depending on the severity of issue.
- Filter 8: Report generation. Gives a webpage report on the severity of the issues along with a summary with each issue found.

A vulnerability scanner at its core is a data transformation pipeline. Vulnerability scanners take raw input and transform it into a report after taking the input through multiple stages until it becomes the output we want. One of the benefits of this pattern is the separation of concerns. Every filter has its own logic and is separated from the rest. This allows for independent testing and no interference between each filter (Shvets, A.). Another benefit includes the ability for filters to be easily added, replaced, or removed. When a new filter is added, only that filter would need to be tested and not the whole scanner itself. This is the type of application that the Pipe and Filter pattern was made for, where the system breaks up the work into multiple steps that are linear.

Strategy Pattern

The strategy pattern is a way to organize different methods into different and separate roles so they can be easily swapped in and out without the rest of the system changing. Each method is written as its own separate class and each follows the same structure so each method could be used interchangeably.

The probe execution stage needs several different types of scans depending on what exactly it is scanning. Rather than putting every method into one block of code, the strategy pattern allows each method to be written as its own separate unit that can be called when needed.



The ScanStrategy method allows for a selection of a specific scan. The PortScan method would scan for open ports to find what services or software versions the system is running on, then compares the findings to a database of known vulnerabilities to flag what might be at risk. The WebAppScan method would send requests to the web app to test for vulnerabilities such as code injections and cross-site scripts. The scanner then analyzes the findings to find any vulnerabilities. The NetworkScan method connects to the server of the service and inspects the encryption setup. It checks for weak or outdated encryptions and verifies the certificate to see whether or not it is valid. The method generally checks for common security protocols and whether they are put in place. Lastly, the CVEScan method looks up known vulnerabilities that are publicly available for the target system and gives a list of weaknesses found along with their severity levels.

Since vulnerability scanning isn't just one singular algorithm, multiple methods being needed is necessary. These four scans; port, web application, network, and CVE each have a completely different procedure. The only similar output they produce is the report of discovered vulnerabilities. Due to the fact that multiple algorithms are being utilized, the strategy pattern works very well for this type of problem. Each target would need different methods of penetration testing and using this design pattern, it allows for a variation of methods, each in their own class. This would prevent one big block of code and instead prioritize organization and efficiency. Any new scan types can be added in and changed at runtime without changing the original code (kartik, 2026). Each scan type has its own responsibility making each method small and simple. This allows for easy, independent testing, along with flexibility.

Adapter Design Pattern

The Adapter design pattern would further improve the flexibility and long term maintainability of the vulnerability scanner. Vulnerability scanners often rely on multiple third party services. APIs, and external data formats that may not share the same structure or communication methods. The Adapter pattern acts as a translation layer between incompatible interfaces. allowing the scanner to communicate with external systems without modifying the core application logic.

Within the vulnerability scanner architecture, adapters would primarily be used for integrating external vulnerability databases, scan tools, and reporting formats. Different APIs such as the National Vulnerability Database (NVD), Common Weakness Enumeration (CWE), or future third party security feeds may return vulnerability information in different formats and structures. Instead of forcing the scan engine to directly handle each unique format, adapter components can standardize the incoming data into a unified internal structure before it is processed by the application.

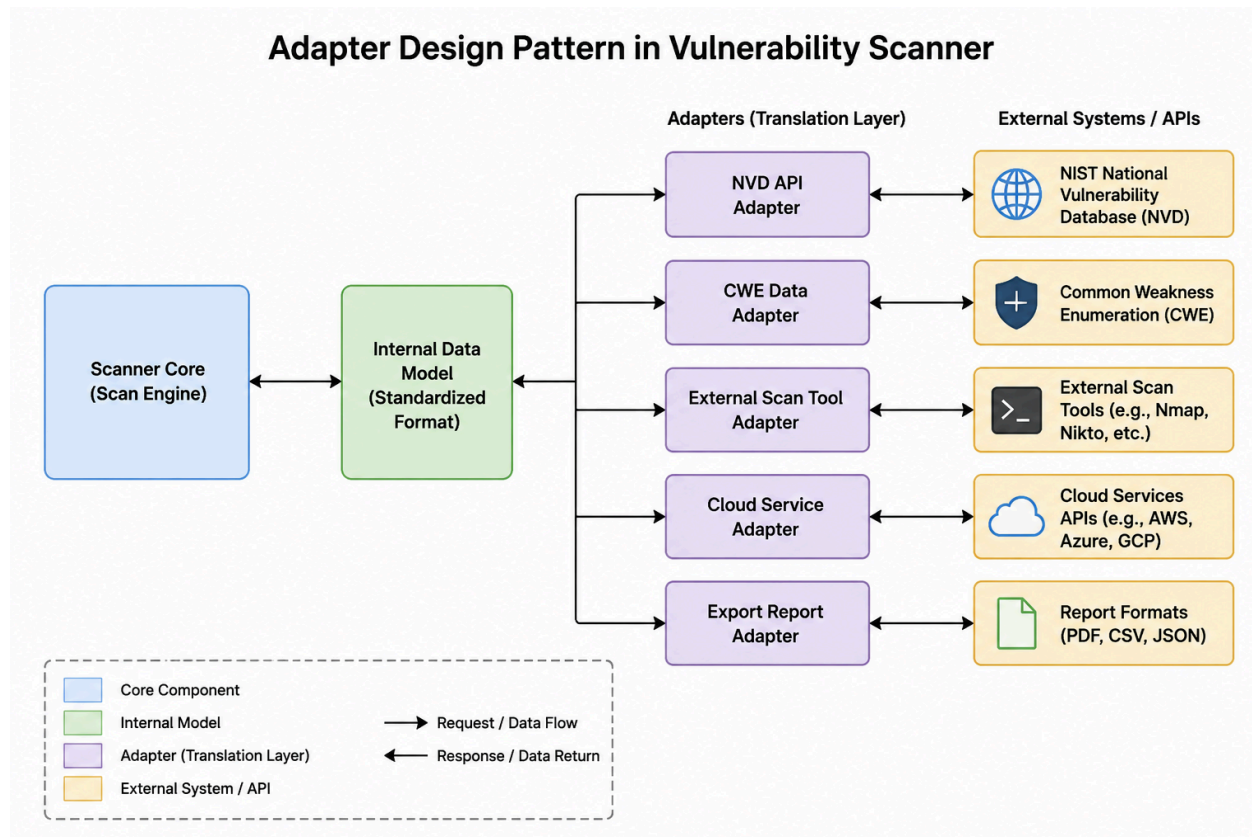
This pattern also improves the scalability because new integrations can be added without changing the core scanning logic. For example, if the scanner later integrates with external penetration testing frameworks, cloud security APIs, or enterprise logging systems, developers would only need to implement a new adapter rather than rewrite the existing architecture. This separation reduces coupling between components and simplifies future development.

The Adapter pattern additionally improved maintainability and testing. Since each integration is isolated into its own adapter module. failures or API changes from one external service are less likely to affect the rest of the application. Individual adapters can also be tested independently to ensure compatibility with external systems before deployment into the main scanner

environment.

Examples of potential adapter integrations include:

- NVD API Adapter
- CWE Data Adapter
- Export Data Adapter
- External Scan Tool Adapter
- Cloud Service Security Adapter



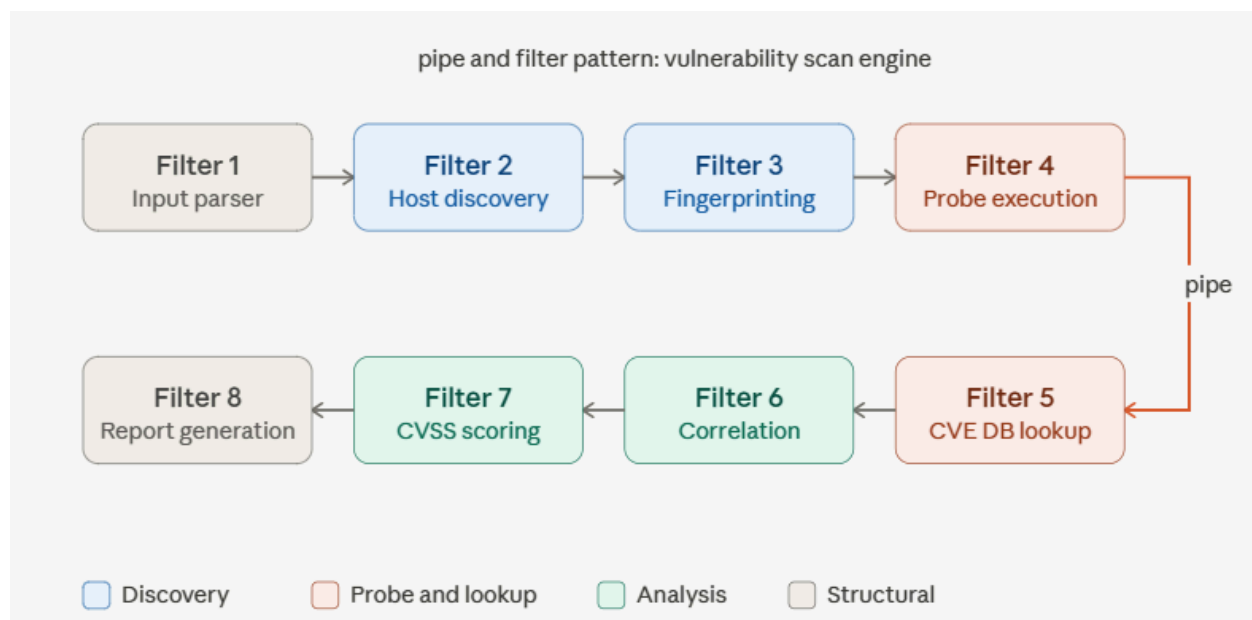
Architecture and Design - Daniel Lopez Section

The client server architecture, which the team established as the primary structural pattern, directly addresses the separation of concerns between the user-facing interface and the computational backend. From a practical standpoint, this matters because the scan engine performing TCP/UDP port enumeration and CVE matching is resource-intensive, while the dashboard presenting results is lightweight. Placing these responsibilities on the same process would create a poor user experience, where the interface becomes unresponsive during active

scans. The client-server separation means the scan engine runs continuously on the server regardless of what the client is doing, and the dashboard simply requests and displays whatever data the server has ready. This maps cleanly onto Use Case 8, where real-time scan progress monitoring requires the server to be pushing status updates while the user is actively watching the dashboard, without blocking either side.

One of the less obvious but important benefits of this architecture for the vulnerability scanner specifically is how it supports the CVE and NIST API integration. The external databases used for vulnerability matching, NIST NVD and CVE, impose rate limits on how frequently they can be queried. Centralizing the API calls on the server side means the system can implement request caching and throttling in one place, rather than having each client independently querying the external APIs and potentially hitting those rate limits simultaneously. This is directly relevant to Use Case 5, where pulling CVE and NIST data via API is a core system function. Without a centralized server managing these requests, the system would risk hitting rate limits and returning incomplete vulnerability data, which would undermine the entire purpose of the tool.

The three tier layered pattern applied to the server side provides the internal organization that the client-server architecture alone does not. While client-server defines where computation happens, the three-tier pattern defines how computation is organized within the server. The API layer acts as the single entry point for all client requests, validating inputs and routing them to the appropriate business logic. The business logic layer is where the scan engine, CVE matching logic, and CVSS risk scoring live. The data access layer manages all reads and writes to the scan history database and the local CVE cache. This separation is critical for maintainability. When the NIST database changes its API format, which has happened historically, only the data access layer needs to be updated. The scan engine and the client interface continue operating without modification. This kind of isolation between layers directly reduces the cost of future changes, which matters for an open-source project where contributors may need to update one component without fully understanding every other part of the system.

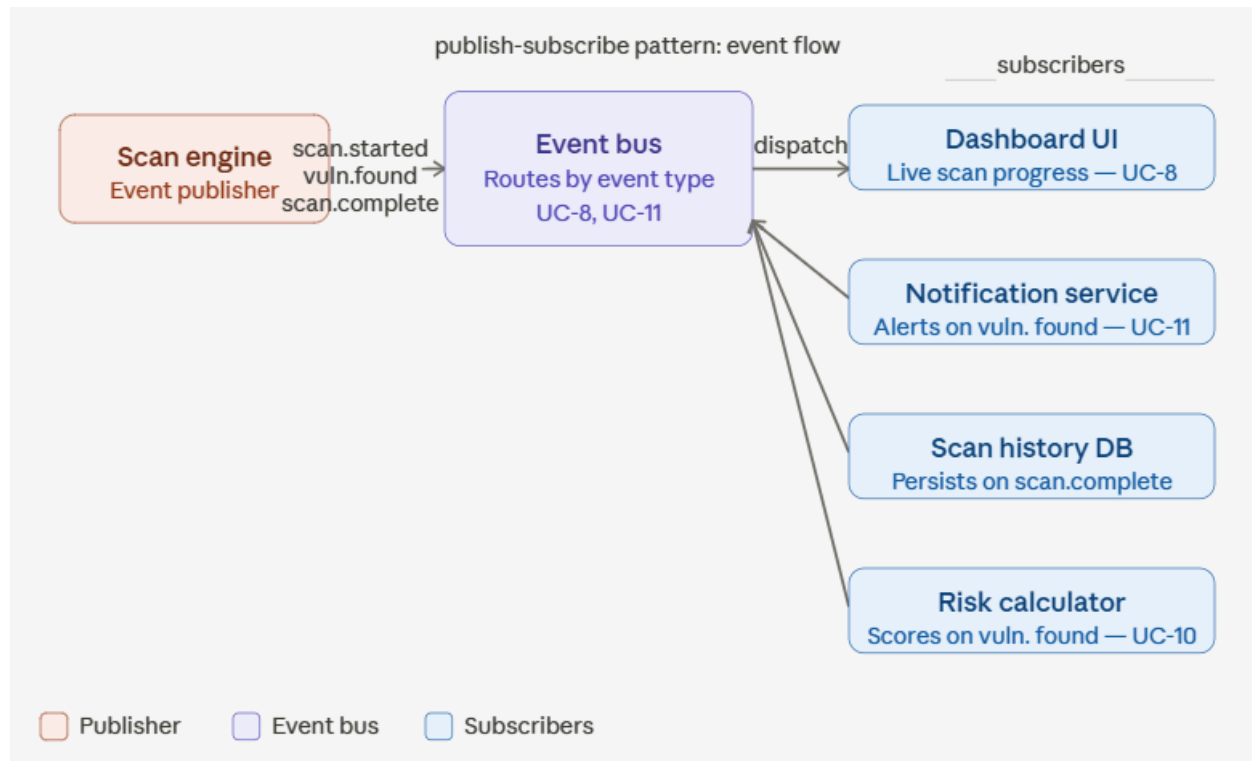


The Repository design pattern, applied within the data access layer, takes this isolation one step further. Rather than having the business logic layer write raw database queries, it interacts with repository interfaces that abstract away the underlying storage mechanism. What this means in practice is that the scan engine does not need to know whether scan history is stored in SQLite, PostgreSQL, or any other database. It calls a method like `getScanHistory()` through the repository interface and receives results. This abstraction makes testing significantly easier, since the repository implementation can be swapped for a mock during unit testing without changing any of the scan engine code. It also means that if the project migrates to a different database in CPSC 491, the change is contained entirely within the repository implementation layer and does not cascade into the business logic.

The Publish Subscribe pattern addresses a problem that neither the client-server architecture nor the repository pattern was designed to handle, specifically the need for event-driven communication within the system. During an active scan, multiple things need to happen simultaneously. The dashboard needs to receive progress updates, the notification service needs to fire when a critical vulnerability is detected. The scan history database needs to be updated incrementally rather than all at once when the scan completes. A traditional request-response model would require the client to continuously poll the server for updates, creating unnecessary network traffic and latency. The Publish-Subscribe pattern decouples the producers of these events, the scan engine, from the consumers, the dashboard and notification service, so that each new event type can be added or removed without modifying the scan engine itself. One small drawback to this design pattern is the added complexity of having a message broker to monitor and develop. Due to this drawback, it's strongly recommended to implement this after the major goals and requirements of the software are met. This is a meaningful design win for a tool that will continue to evolve through CPSC 491.

The MVC pattern on the client side completes the picture by providing structure to the user-facing layer. The Model holds the current scan configuration and result state locally on the client. The View renders the dashboard, results table, and vulnerability detail panel. The Controller handles user interactions such as initiating a scan, applying filters to results, or triggering a report export. Keeping these three concerns separated means that changes to how the dashboard looks do not require changes to how scan actions are processed, and vice versa. This matters particularly for the personas identified in the UX Design section. The interface for a novice user like Sam, the small business owner, may evolve differently from the interface for an expert user like Alex, the IT administrator. Because the View layer is decoupled from the Controller and Model, different interface configurations can be developed for different user contexts without rewriting the underlying logic.

Taken together, these patterns form a coherent design rather than an arbitrary collection of buzzwords. The client-server architecture defines the physical separation of concerns. The three-tier layered pattern organizes the server internally. The Repository pattern abstracts data access within the data layer. The Publish-Subscribe pattern handles asynchronous event communication across components. The MVC pattern structures the client. Each pattern operates at a different layer of the system and solves a different class of problem, and that intentionality is what makes the architecture appropriate for a project of this scope and audience.



UX Design

The application uses a security operations dashboard aesthetic. The goal is not to make the interface flashy, but to make it feel familiar to users who have seen SIEM tools, vulnerability management platforms, or system monitoring dashboards. A dark background is used because security dashboards often display large amounts of technical information for long periods of time. The darker layout also helps security colors stand out clearly.

The visual language is built around urgency and readability. Critical vulnerabilities are shown with red indicators, high findings use orange or yellow indicators, successful actions use green, and information items use blue or gray. This gives users a quick way to understand risk before reading every detail. The interface should allow a user to glance at the dashboard and immediately understand whether the scan found something serious.

The design avoids unnecessary decoration. Most of the screen space is reserved for scan status, affected hosts, vulnerability, severity, CVE identifiers, and recommended actions. This matches the purpose of the scanner because users are not opening the tool for entertainment. They are using it to make decisions about risk, patching, reporting, and system exposure.

Aesthetic

When it comes to the experience of the user and design of the scanner, we want to deliver a smooth, easy to navigate experience for the user. We also want the scanner's aesthetics to be easy on the eyes and pleasant to look at. It is important for the user to want to have a pleasant experience when using the tool and for the user to want to return to the platform for future use.

The Vulnerability Scanner adopts a dark-professional aesthetic, prioritizing clarity, data density, and visual hierarchy over decorative elements. The color language maps directly to security severity, red for critical, orange for high, yellow for medium, and green for low. Which aligns with established conventions in cybersecurity tooling (CVSS score displays, SIEM dashboards, etc.). This makes scan results scannable at a glance without requiring the user to read every line. Typography is kept clean and monospace-influenced in data fields to reinforce a technical, trustworthy tone. The overall aesthetic draws from the "utility-first" design philosophy seen in tools like Nmap's GUI frontends and Burp Suite functional density without visual clutter.

The vulnerability scanner further expands on this utility-focused aesthetic by emphasizing readability and rapid interpretation of technical information. Since the application is intended to present large amounts of scan data, vulnerability findings, CVE identifiers, and risk summaries simultaneously, the interface prioritizes clarity over decorative interface elements. The high contrast design ensures important findings immediately stand out to the user, especially during active monitoring sessions where rapid interpretation is critical. This visual structure helps both experienced analysts and less technical users identify the most important information without becoming overwhelmed by excessive detail.

The interface also follows a minimal and highly structured layout designed to reduce cognitive overload. Many vulnerability scanners currently available overwhelm users with dense outputs and poorly organized technical information. The goal of this scanner is to maintain a professional and technically focused appearance while still remaining approachable for beginner users and students learning vulnerability assessment workflows. Monospaced and code-style fonts are consistently used for IP addresses, CVE IDs, port numbers, and service information in order to visually separate machine-readable data from descriptive explanations and remediation guidance.

The dashboard design strongly emphasizes information hierarchy and workflow accessibility. Frequently used actions such as "New Scan," "Reports," "Scan History," and "Settings" remain persistently accessible through the left-side navigation panel, while the center section dynamically updates based on the user's active task. Important vulnerability findings, live scan updates, and risk summaries are prioritized visually to reduce unnecessary navigation during active use. The design intentionally avoids excessive animations or overly complex visual effects because the application is intended to function as a productivity and analysis platform rather than a consumer-oriented application.

The aesthetic being used for the vulnerability scanner would be a simple dark themed, high contrast aesthetic. This aesthetic would be best used for this type of application since it would prioritize giving important information to the user and doesn't use excessive designs. Since the vulnerability scanner is a tool and not a consumer product, it should stay formal, simple, and easy on the eye.

User Research

Understanding who will use this tool requires thinking about the range of people who perform vulnerability assessment in practice.

For further understanding, we plan to conduct a series of research methods during development to further understand how we can tailor our program's features to user needs. According to the Nielsen Norman Group, some of the most effective research methods to use throughout development include field studies which would involve analyzing their activity to understand the context of product use, interviews with users to further discuss the topic project need to evaluate feedback directly, and raw benchmarking to study the quantitative data such as how much users actually engage with the scanner or the exporting of data. This creates a strong research foundation that allows us to work closely with user expectations of the product and gives us a clearer view of design problems that may exist.

The expected users for this application were identified by looking at who would realistically need a lightweight vulnerability scanner. The main users are security analysts, IT administrators, students, developers, and small business owners. These groups do not all have the same level of technical knowledge so the interface cannot assume that every user understands CVE scoring, port scanning, service enumeration, or remediation planning.

Security analysts need fast access to technical findings. They care about affected hosts, severity, CVSS scores, CVE identifiers, and remediation steps. IT administrators need clear summaries because they may be responsible for patching systems, but may not specialize in Vulnerability research. Students need explanations that help them learn what scan results mean. Small business owners need plain language summaries because they may only want to know whether their system is exposed and what action to take next.

This range of users shaped the UX decisions. The dashboard provides technical details for advanced users, but also includes summary panels, severity labels, and remediation guidance for less experienced users. Nielsen Norman Group explains that interfaces should match the language and expectations of users rather than forcing users to understand internal system terminology. This supports the decision to combine technical CVE data with plain language explanations. Anyone with technical knowledge would be able to use the vulnerability scanner. The main type of user of a vulnerability scanner would be employees working in the IT department of a business (Siddiqui, 2023). Other users include security analysts and software developers. All types of users have the same similar goal. It is to ensure the tools and systems used in their company are safe from any malicious cyberattacks. IT administrators would be responsible for maintenance on their company's networks and servers. This could also include web servers, routers and operating systems (Lim, 2021). Security analysts would also be a major user of the tool. Their job is to find vulnerabilities and patch them for the company they work for. Lastly, software developers would also benefit from the use of a vulnerability scanner. They may not go into depth on their security report because they may just be using the scanner to ensure if a tool they are using is indeed safe to add into their application. The use of this scanner would significantly improve the security of any business' assets and would be highly beneficial for employees working in these roles.

Security Analyst

One of the primary users of Vuln Scanner would be information security analysts. The information security analyst roles bridge the gap between business and technical teams. This role performs vulnerability and threat hunting, systems network and maintenance, designing and implementing data governance, and ensuring compliance with laws and regulations(cite bls). These individuals go out and hunt for threats that possibly breach their system and configure protection strategies preventing these attacks. An overload of technical input is one common issue found in most vulnerability scanners on the market. Many of the findings generated in a report may contain less important details adding difficulty in finding critical vulnerabilities. The analyst also needs a dashboard containing scan configuration, live monitoring, and organized results. The UX of our scanner aims to hit both of these needs through our utility first design philosophy mentioned in the aesthetic portion of the paper. We chose to use a high contrasting theme with detection colors that would be easily noticeable by analysts. This allows for the most important information to be noticed easily and immediately. The basis of the Vuln Scanner would ultimately be another tool in their toolbox for these analysts to use in solidifying their defenses.

Students

Another user that would benefit from using Vuln scanner would be individuals learning about the field better known as students. Given that our scanner is an open source project, Vuln grants easy access to individuals who normally wouldn't be able to afford an industry level scanner. Students would be able to learn how scanning detects open ports, understanding CVE/NVD data connected to services, interpreting vulnerability reports and running scans themselves. A hypothetical workflow for a student would look like the following:

1. Enter a target IP or host
2. Configure Scan parameters
3. Run the scan
4. View open ports and detected services
5. Match findings to known vulnerabilities
6. Interpret the final report.

This demonstrates to the student the true workflow of a vulnerability scan. Due to its beginner-friendly nature, students benefit from our scanner's safe scanning intensity options, authorized targeting, and warnings before scanning. The open-source nature of the scanner also allows students to learn about software architecture and design patterns. Students can study how programs function cohesively and how to implement these design choices so they work in unison.

Small Business Technical Staff

Small business staff can use this lightweight scanner as an entry level, easy to understand security checker for company devices and services. These types of users are unlikely to have advanced cybersecurity experience; the application provides a simple workflow: enter a target, selecting a scan type, reviewing severity-based results, and exporting reports. This scanner can be used on all company approved devices that include but are not limited to office computers, file servers, web servers, etc. The scanner helps smaller businesses that do not have a budget for this type of software to identify exposed services, outdated software, and potential security risks before they

can escalate. By presenting vulnerabilities with plain language descriptions, these users can find remediation steps where the users if needed can get further help.

Rather than targeting a single user type, we considered the spectrum from security novices to experienced professionals. Research into existing scanners (OpenVAS, Nuclei, OWASP ZAP) revealed a consistent usability gap. Most tools are either too complex for non-specialists or too limited for professionals who need customizable scan options. The Vulnerability Scanner is designed to bridge that gap. The scan interface offers quick, full, and custom scan modes so that a student or small business owner can run a basic scan with one click, while a security professional can configure port ranges, toggle service detection, and export results in PDF or CSV format. This dual audience approach informed every UX decision from layout to labeling.

The research process also considered how users interact with vulnerability scanners during real working conditions rather than only in controlled environments. Security scans are often long running operations, which means users may multitask while monitoring scan progress. Because of this, the interface emphasizes persistent visibility of active scans, elapsed time, risk summaries, and notification indicators. Real time feedback was prioritized because users generally expect confirmation that a process is still active. Without continuous updates, users may incorrectly assume the scanner has frozen or failed. This directly influenced the inclusion of live monitoring panels, progress indicators, and event based notifications throughout the dashboard.

Another important research consideration was reducing cognitive overload for less experienced users while still maintaining technical depth for advanced users. Existing enterprise vulnerability scanners often present extremely dense outputs filled with CVE references, service data, and raw vulnerability details all at once. While experienced analysts may prefer this level of information density, newer users can struggle to determine which findings are most important. To address this issue, the scanner uses progressive disclosure principles by displaying summarized information first and allowing users to expand findings for more technical detail only when needed. This helps maintain readability without sacrificing functionality.

Accessibility and readability also influenced several interface decisions. The system uses consistent color coding, clear spacing between sections, and structured layouts so users can identify critical vulnerabilities quickly. Severity levels are paired with both text labels and color indicators to improve clarity for users who may have difficulty distinguishing certain colors. Navigation is kept consistent across all sections of the application so users do not need to relearn the interface when switching between scan monitoring, reports, or vulnerability details. These decisions were informed by common usability concerns identified in cybersecurity dashboards and technical monitoring software.

The research findings ultimately reinforced the importance of balancing technical functionality with usability. The scanner is intended to function both as a professional security tool and as an accessible learning platform for newer users. As a result, the UX design emphasizes clarity, flexibility, and efficient workflows while still supporting advanced scan configurations and detailed vulnerability analysis.

Personas

Persona 1 - Alex, IT Administrator (Primary User)

Alex manages the network infrastructure for a mid sized company. They run periodic vulnerability scans as part of compliance requirements and need results that are both comprehensive and exportable for audit reports. Alex is technically proficient but is under time constraints, they need a tool that gets to results fast and presents them in a format they can hand to a manager. Alex will use Full Scan mode, CVE lookup, and the export report feature most frequently.

Persona 2 - Jordan, Computer Science Student (secondary user)

Jordan is studying cybersecurity and wants hands on experience with real scanning tools. They are comfortable with command line tools but appreciate a visual interface when learning what scan results mean. Jordan needs clear labels, tooltips or references linked to NIST/CVE databases, and the ability to run against test environments. Jordan will primarily use quick scan and the vulnerability details pop up to understand what each CVE means.

Persona 3 - Sam, Small Business Owner (Tertiary User)

Sam runs a small e-commerce site and has heard about data breaches affecting similar businesses. Sam has no technical security background but wants to know if their web server is exposed. Sam needs a simple entry point, plain-language severity explanations, and clear remediation steps — not just a list of CVE IDs. Sam benefits most from the Risk Summary section and the remediation guidance in the Vulnerability Details panel.

Persona 4 - Alice the Security Analyst

Alice is a junior security analyst at her mid-sized finance service company. She regularly runs vulnerability scans on her company's infrastructure such as the internal company network. Since Alice has been a professional in her field for some time, she is comfortable reading CVE data. She mainly wants to focus on the most critical weaknesses so she can patch them as soon as possible. A severity ranking would make her job much easier and would deliver what she expects from a vulnerability scanner.

Persona 5 - Isaac the IT Administrator

Isaac is a senior IT administrator at a mid-sized e-commerce company. He manages a small team of four people. He does scan systems often, but when he does, he wants reports that go straight to the point so he can hand them to his team to solve the main issues. He does not especially focus on security since he primarily spends his time ensuring that the company systems and accounts have no issues. He doesn't have much time to go into detail about every finding, but needs to know what systems are most at risk. He wants a simple given report that summarizes the findings without an overwhelming amount of information.

Persona 6- Sandra the Software Developer

Sandra is an entry level software developer working at a healthcare technology company. She works with a team building the infrastructure for the patient data management. The security of this information is extremely important since any compromise on patient information could put her company in a lot of legal trouble. She is not very focused on cybersecurity since her career is centered around backend engineering. She does not want to risk any leakages from her own mistakes so she wants to be extra sure that the libraries she uses when writing code is actually safe to use. She sends over her progress to the company's security team and receives the report. She wants to know what libraries are the flagged ones easily and quickly.

Persona 7 - Sanjay the Senior SOC Analyst

Sanjay is a senior at SecTec. A moderately sized firm that works in consulting and security advising for larger companies. He has a new batch of employees, including those who don't have much practical experience with running tests and investigating reports. So he sets up several VMs loaded with intentional misconfigurations and issues for the new employees to run a series of practice scans to find these issues. A lightweight, industry standard scanner loaded into a controlled setting like a VM would be a perfect tool for training the new employees.

Persona 8 - Maya the Compliance Officer

Maya works as a compliance officer for a healthcare organization that must regularly meet security and privacy regulations. She is not directly responsible for performing vulnerability scans, but she reviews reports to ensure systems comply with internal policies and external standards. Maya needs reports that are easy to read, exportable, and organized by severity so she can present findings during audits and meetings. She benefits most from the report generation tools, summarized risk assessments, and historical scan tracking features.

Persona 9 - Ethan the DevOps Engineer

Ethan works as a DevOps engineer at a cloud-based software company. He is responsible for maintaining deployment pipelines, cloud infrastructure, and server uptime. Ethan frequently scans staging and production environments before deployments to identify exposed services or outdated software versions. Since he works in fast-paced deployment cycles, he values automation, quick scan configurations, and real-time monitoring updates. Ethan primarily uses scheduled scans, live scan progress monitoring, and notification alerts to quickly identify security risks before releases go live.

Persona 10 - Priya the University Lab Instructor

Priya teaches introductory cybersecurity and networking courses at a university. She manages a classroom lab environment where students practice ethical hacking and vulnerability assessment techniques in isolated virtual machines. Priya needs a scanner that is simple enough for beginners to learn while still exposing students to industry-standard concepts such as CVE databases, severity rankings, and scan reports. She values the scanner's educational usability, guided interface, and ability to safely demonstrate real vulnerability assessment workflows within controlled environments

Persona 11 - Lena the Managed Service Provider (MSP) Technician

Lena works for a managed IT services company that supports multiple small businesses at once. She is responsible for monitoring and maintaining dozens of client systems ranging from local retail stores to professional offices. Since she handles many environments simultaneously, Lena needs a vulnerability scanner that can quickly organize scan results by client, prioritize the most severe findings, and generate reports that are easy to share with customers who may not have technical expertise. She values centralized scan management, fast report exports, and clear remediation recommendations so she can efficiently support multiple clients without spending excessive time analyzing each individual scan.

Persona 12 - Marcus the System Administrator for a School District

Marcus manages the IT infrastructure for a public school district consisting of multiple campuses, computer labs, and administrative offices. He is responsible for maintaining secure systems across hundreds of devices while working with limited budgets and aging hardware. Since educational environments often lack dedicated cybersecurity teams, Marcus needs a vulnerability scanner

that is lightweight, easy to deploy, and capable of identifying critical vulnerabilities quickly without requiring extensive manual configuration. He values centralized scan management, scheduled scans, and simple remediation guidance that helps him prioritize the most urgent risks affecting student and staff systems

Frameworks

The application follows two primary UX frameworks. First, Nielsen's 10 Usability Heuristics guide the interface design, particularly "visibility of system status" (scan progress feedback), "match between system and the real world" (plain-language severity labels alongside technical CVE IDs), and "error prevention" (confirming target IP before launching a scan). Second, Goal-Directed Design (GDD) as defined by Alan Cooper shapes how the interface is structured. Each screen is organized around what the user is trying to accomplish (configure a scan, review results, understand a vulnerability, export a report) rather than around the system's internal architecture. This ensures the interface feels task-oriented rather than tool oriented.

In addition to these frameworks, the application also incorporates principles from Information Architecture to improve navigation and content organization. Vulnerability scanners often contain large amounts of technical information, including scan histories, CVE references, reports, risk summaries, and live monitoring data. Without a clear organizational structure, users can become overwhelmed or lose track of important findings. To reduce this issue, related features are grouped into predictable sections within the interface. Scan management, reporting tools, vulnerability analysis, and settings are separated into consistent categories so users can quickly locate the functionality they need without unnecessary searching.

The scanner also applies feedback-oriented interaction principles throughout the interface. Actions such as starting scans, exporting reports, or filtering vulnerabilities provide immediate visual confirmation so users understand that the system has received their input. This reinforces usability by reducing uncertainty during longer operations. The combination of usability heuristics, Goal-Directed Design, and structured information architecture creates an interface that remains approachable for newer users while still supporting advanced workflows for experienced analysts.

The UX design also uses Nielsen's usability heuristics as the main evaluation framework. This framework fits the project because the scanner must communicate system status, prevent configuration mistakes, and help users recover from errors. Nielsen Norman Group identifies principles such as visibility of system status, consistency and standards, error prevention, recognition rather than recall, and helps users recognize and recover from errors. These principles apply directly to a vulnerability scanner because scans can take time, errors can happen during configuration, and users need clear feedback.

Visibility of system status is handled through live scan progress, current scan stage, elapsed time, and scan completion indicators. Error prevention is handled through input validation for IP addresses, port ranges, and scan type selection. Recognition rather than recall is handled by placing CVE descriptions, severity labels, and remediation guidance directly inside the results screen instead of expecting users to search for that information separately.

The project also uses personas as a UX planning tool. Personas help the team design around realistic user needs rather than designing only for themselves. Nielsen Norman Group describes

personas as realistic representations of target users that support user centered design and help teams keep user groups memorable. This is important for this scanner because the needs of a student, a small business owner, and a security analyst are different.

Goal-Directed Design

The Goal-Directed Design is a design framework created by Alan Cooper that prioritizes what the user is actually trying to achieve, instead of what the system can deliver. When it comes to delivering this framework, you must first study your users and what their needs are. Observe the user's needs and understand what their end goal is. Before designing a program and writing code, understand what the user wants and base your modeling around that. The use of detailed personas, such as a security analyst or a software developer not too familiar with cybersecurity concepts, can further help the design and requirements of the product. The team could add features that assist potential users with different features, language, and an interactive interface. This ensures that the design of the application doesn't just work, but also assists all types of users comfortably. The design made should be tested and adjusted based on feedback to ensure the best experience possible before deployment of the application. The Goal-Directed Design would be a great framework for the vulnerability scanner since user feedback would assist in making a better platform for future users, and creating a tool that is effective and accessible for all types of users.

Information Architecture

Information architecture prioritizes the need to make information easily accessible and easy to understand. The organization and structure of this architecture puts understanding the given information as its priority. The architecture focuses on three main points: who the users are, the context of why they are using the tool, and the content they want to engage with (What is information architecture?). This approach makes it that the design is driven by actual users' needs instead of technical convenience. This ensures that the system feels more accessible to the user interacting with it. Systems should have categorizations, such as severity of weakness found. These categories would use established frameworks such as the CVSS (Common Vulnerability Scoring system) which gives users a clear summary of the vulnerabilities found, along with how much of a priority it should be. The user does not need a high level of cybersecurity knowledge to understand the urgency of each vulnerability. Navigation should be easily usable to ensure efficiency. This means that clear labeling, consistent structures, and groupings can assist a user in finding something they need with no confusion.

Outside of navigation, information architecture also leverages findability. For this vulnerability scanner, it allows for features like filters and sorting. Key word searches also allow for easier findability. If a user is unable to find what they are looking for quickly, then the user satisfaction would be diminished.

Another key feature of information architecture is labeling systems. Using nontechnical language could help reduce the complexity of the vulnerability scanner for users who don't have a deep understanding of cybersecurity. Instead of the user being presented with raw CVE data, the scanner would give descriptions of each vulnerability in a more readable format.

This architecture framework would be great for the vulnerability scanner since it prioritizes user comfort and needs. By building the scanner with the users in mind, the scanner would be a tool that doesn't just deliver what it's meant to, but also delivers an easy to use interface for any type of

user, ranging from security analysts to software developers using a vulnerability scanner for the first time.

Architecture and Design Patterns

The UX architecture follows a Dashboard Layout Pattern, a persistent navigation bar at the top (New Scan, Scan History, Reports, Settings) combined with a left-side input panel and a main content area for results. This pattern is appropriate because users need constant access to core functions while reviewing potentially large result sets. The Progressive Disclosure design pattern is used for vulnerability details, the main results table shows only the essential columns (port, service, version, risk level, CVE count), and full details (CVSS score, description, remediation, NIST reference) appear in a pop up panel only when the user explicitly requests them. This reduces cognitive load for users reviewing many results at once. Navigation follows a Hub-and-Spoke model, the scan results page is the central hub, with spokes leading to detailed vulnerability views, report export, and scan history, all of which return the user to the hub.

The interface also incorporates Card-Based UI patterns throughout the dashboard to separate scan summaries, vulnerability alerts, and monitoring widgets into visually distinct sections. This improves readability by preventing information from blending together, especially when multiple scans or vulnerabilities are displayed simultaneously. Each card acts as a contained unit of information with its own controls and status indicators, allowing users to process information more efficiently during active monitoring sessions.

Another important UX design decision is the use of consistency and persistence across navigation elements. Core actions such as initiating scans, reviewing reports, and accessing vulnerability details remain accessible regardless of which screen the user is currently viewing. This minimizes navigation confusion and supports fast workflows for users who repeatedly move between monitoring and reporting tasks. Consistent placement of interface elements also reduces the learning curve for newer users because controls behave predictably throughout the application.

The architecture also supports scalability from a UX perspective. As additional scan types, reporting tools, or notification systems are introduced in future iterations of the project, the dashboard layout and hub-and-spoke navigation model allow these features to be integrated without significantly changing the existing interface structure. This helps preserve usability over time while supporting future expansion of the scanner's capabilities.

The UX architecture is organized around a dashboard centered workflow. After login, the dashboard becomes the main navigation point for the application. From there, the user can start a scan, view scan progress, open previous scan results, review vulnerability details, generate a risk summary, or export a report. This structure keeps the most important actions close together instead of scattering them across unrelated screens.

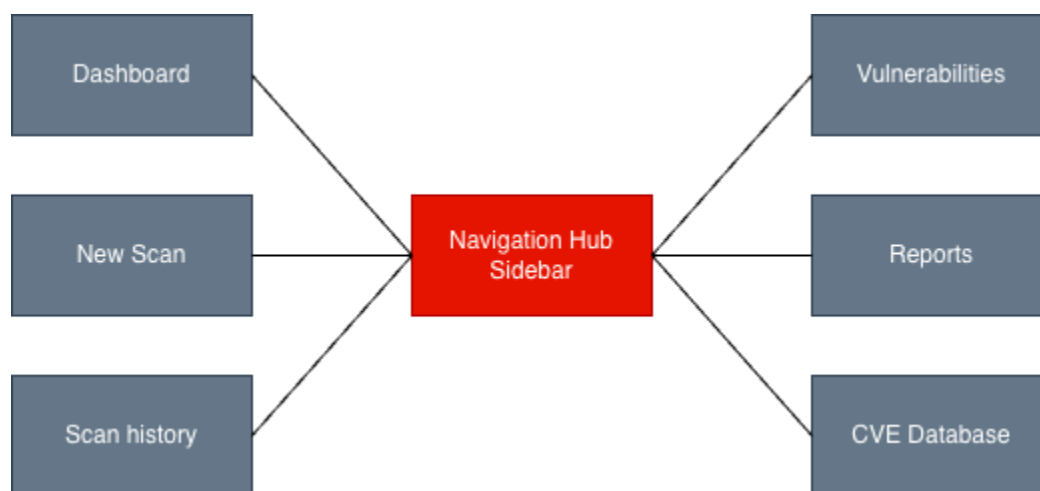
The interface follows a progressive disclosure pattern. Basics users first see the most important information, such as total vulnerabilities, severity breakdown, overall risk, and recommended next steps. More technical details, such as CVE identifiers, affected ports, service versions, and database references, are available when the user expands or opens a specific finding. This allows the tool to serve both beginner and advanced users without overwhelming either group.

The dashboard also uses a card based layout. Each scan result or vulnerability can be displayed as a card containing the host, port, service, severity, CVE number, and remediation guidance. This makes the results easier to scan than a long block of text. It also supports filtering and sorting by severity, host, or scan date.

Hub-and-Spoke Pattern

The Hub-and-Spoke architecture pattern is a useful pattern that has the entire directory of a system put in a left hand bar. This left hand bar is also known as the 'hub' and allows for easy navigation between different areas known as a 'spoke'. This architecture pattern benefits in security, cost, scalability, and visibility. With security, putting navigation as a single hub makes access control and authentication can be enforced at one point instead of being on multiple sections. This would reduce the risk of unauthorized access. The sections being centralized allows the user to not get lost while navigating the scanner and allows for stronger security. This allows for a good, smooth experience for the user (Rogers, 2026). The spokes in the scanner include the dashboard, new scan, scan history, vulnerabilities, reports and CVE database

Each spoke has a specific purpose within a user's way of work. A dashboard allows for a good high-level overview of the user's activity and history. This allows for the user to keep track of their results over time. The reports can be used as a way of communication to the user's stakeholders, while the vulnerability database results are used as reference to the issues found. With the navigation hub sidebar, the user can easily navigate through the scanner and find all the spokes needed. This is because each spoke is one click away from the hub, which makes it simple for users to move from spoke to spoke, such as the scan history page and the CVE database without losing where they originally were on the web page. This allows the user to move comfortably throughout the tool, which allows for more efficiency.



Progressive Disclosure Design

The main objective of this design is to assist users through a highly technical tool by only giving the user the information they need (Ontañón, 2024). This is because of the cognitive load theory, which describes how people have a limited amount of information they could process. By showing

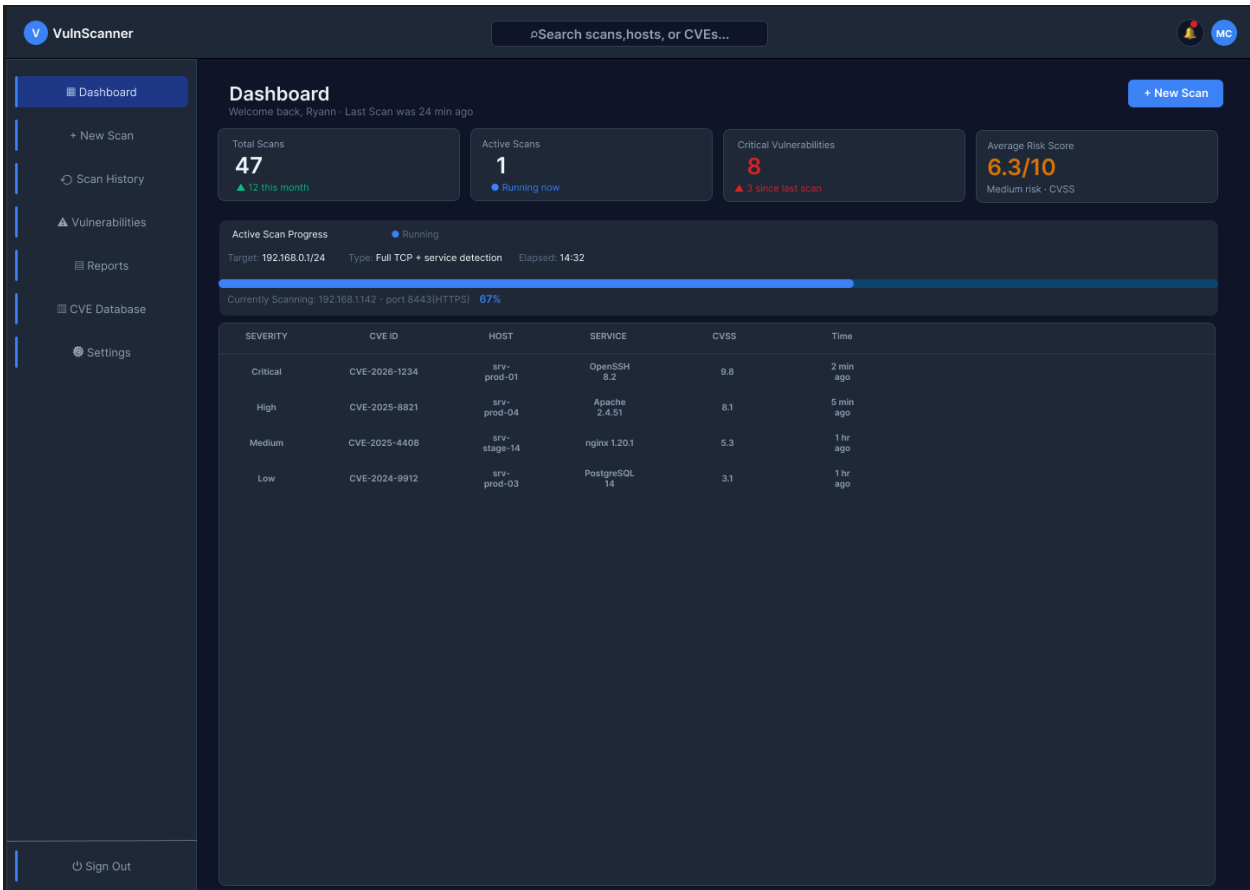
users what's relevant first, reduces the stress and load you could give on a user. This allows for users to make better decisions on their end and make them feel overall comfortable. This design helps users see what's most important once they are in a section of the vulnerability scanner. This means that the vulnerability scanner would present informational summaries, such as vulnerabilities found, the severities, and most critical issues, first before going deeper into each finding.

Some users do not want to see raw CVE data, but instead want a more simplistic summary. A software developer would be a user that would not want a lot of details, but a security analyst would want more information for further assistance. A software developer scanning their application needs to know what needs to be fixed and how to fix it, while a security analyst would want to conduct a formal audit, which would require all details and technical information, such as CVSS scores and remediation fixes, to do a full organizational risk. The progressive disclosure design would allow the security analyst to click a specific CVE and read more into the information.

Putting information in layers supports better decision making for the user. The first layer would show the vulnerability name, how severe it is, and the component it affects. The second layer would show the whole CVE description, CVSS string, and other information after a click. The third layer would show remediation strategies and other links regarding the specific vulnerability. Each layer would give the user information at a load that does not overwhelm them. The more information the user needs, the user could click a button that gives them more information. This allows for a user to do just a quick, brief check, to a whole analysis. Thus, the user does not get overwhelmed with information, but all users can comfortably use the same tool.

This inclusivity is beneficial for both the users and the developers. Rather than making multiple interfaces for different users, progressive disclosure allows the vulnerability scanner to be able to serve all types of users through just one interface. In conclusion, the tool would feel simple and easy to use for a beginner, but powerful and efficient for a technical expert.

Mock Up Photo



Prototype

The prototype demonstrates the basic flow of the vulnerability scanner before the full application is implemented. It is not intended to perform real scanning. Instead, it shows how a user would move through the system, what screens they would see, and how the application would respond to user interaction. The prototype begins with a login screen. After entering sample credentials, the user moves to the dashboard. The dashboard displays a summary of recent scans, active scan status, vulnerability severity totals, and alert notifications. From the dashboard, the user can select a new scan workflow.

The scan configuration screen allows the user to enter a target IP address, choose a scan type, select a port range, and begin the scan. Once the scan is started, the prototype moves to a live

scan screen. This screen shows a progress bar, current scan stage, discovered services, and early vulnerability findings.

After the scan is complete, the prototype moves to a results screen. The results screen displays sample CVE findings, severity levels, affected services, and remediation guidance. The user can then open a risk summary page or export the results. This flow demonstrates that the prototype responds to clicks and moves from one screen to another, which matches the assignment requirement for an animated or interactive prototype.

Figma is a suitable tool for this prototype because it allows screens to be connected into interactive flows. Figma's prototyping tools are specifically designed to show how users move through screens and interact with designs before development begins.

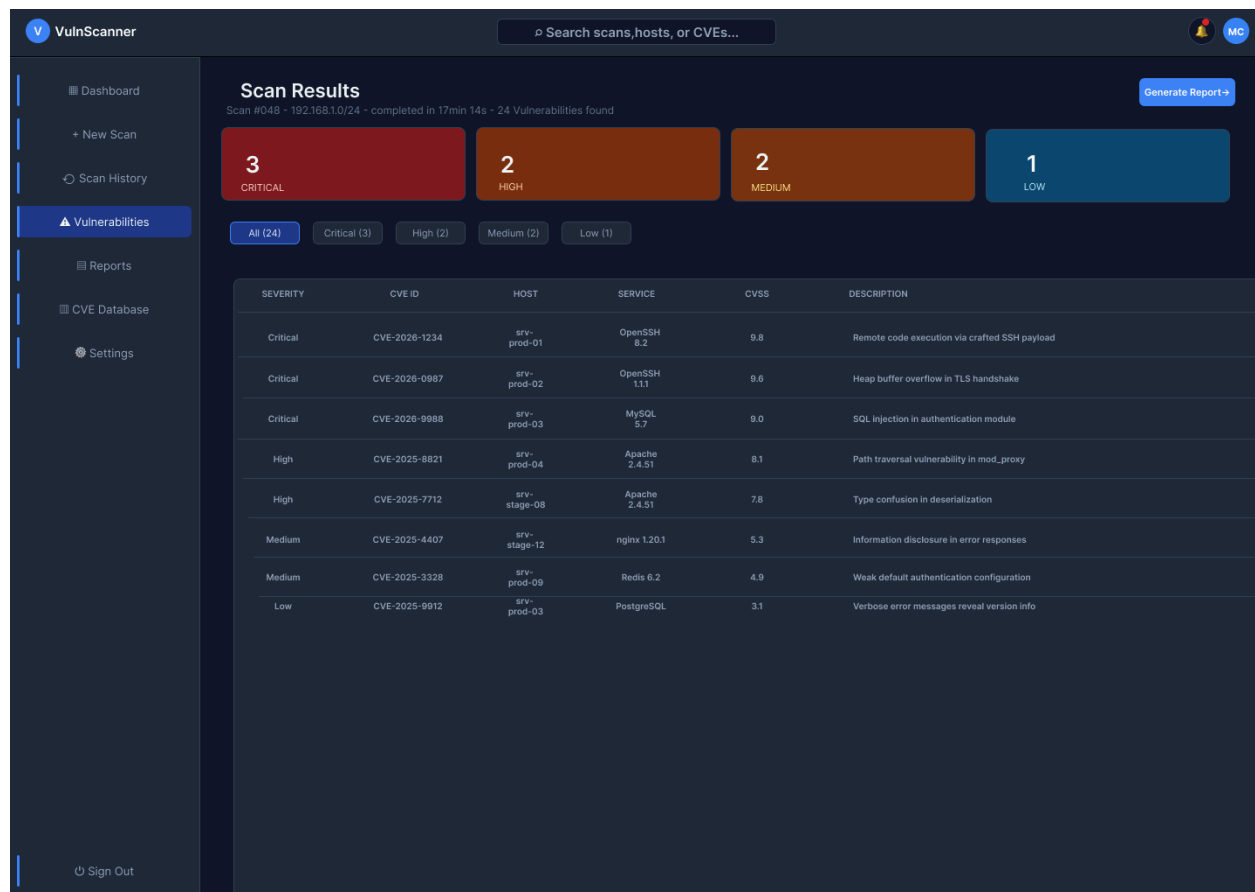
The prototype was designed to simulate the order in which a real user would interact with the vulnerability scanner during a normal workflow. Rather than presenting isolated pages, the prototype connects screens together so the user can move through the application in a realistic sequence. This interaction flow begins with authentication, transitions into scan setup, continues into active monitoring, and finishes with vulnerability review and report generation.

Special attention was given to the transition between screens because scan operations are not instantaneous. During the simulated scanning phase, the interface updates gradually to imitate how a real scan engine would behave while discovering hosts, identifying services, and comparing results against vulnerability databases. This gives users feedback that the system is actively processing information rather than appearing frozen or unresponsive.

The prototype also demonstrates how the interface changes depending on the current stage of the scan lifecycle. Before the scan begins, the interface prioritizes configuration controls and validation feedback. During scanning, the dashboard shifts toward progress monitoring and live status updates. After scan completion, the interface reorganizes itself around vulnerability analysis, severity categorization, remediation guidance, and export functionality. This shifting interface structure helps reduce unnecessary information on screen and keeps the user focused on the task most relevant to the current stage of operation.

Although the prototype uses simulated data rather than real vulnerability results, the workflow reflects how the final application is intended to function during CPSC 491 implementation. The purpose of the prototype is to validate navigation structure, usability, and user interaction patterns before backend development begins.

Scan Results

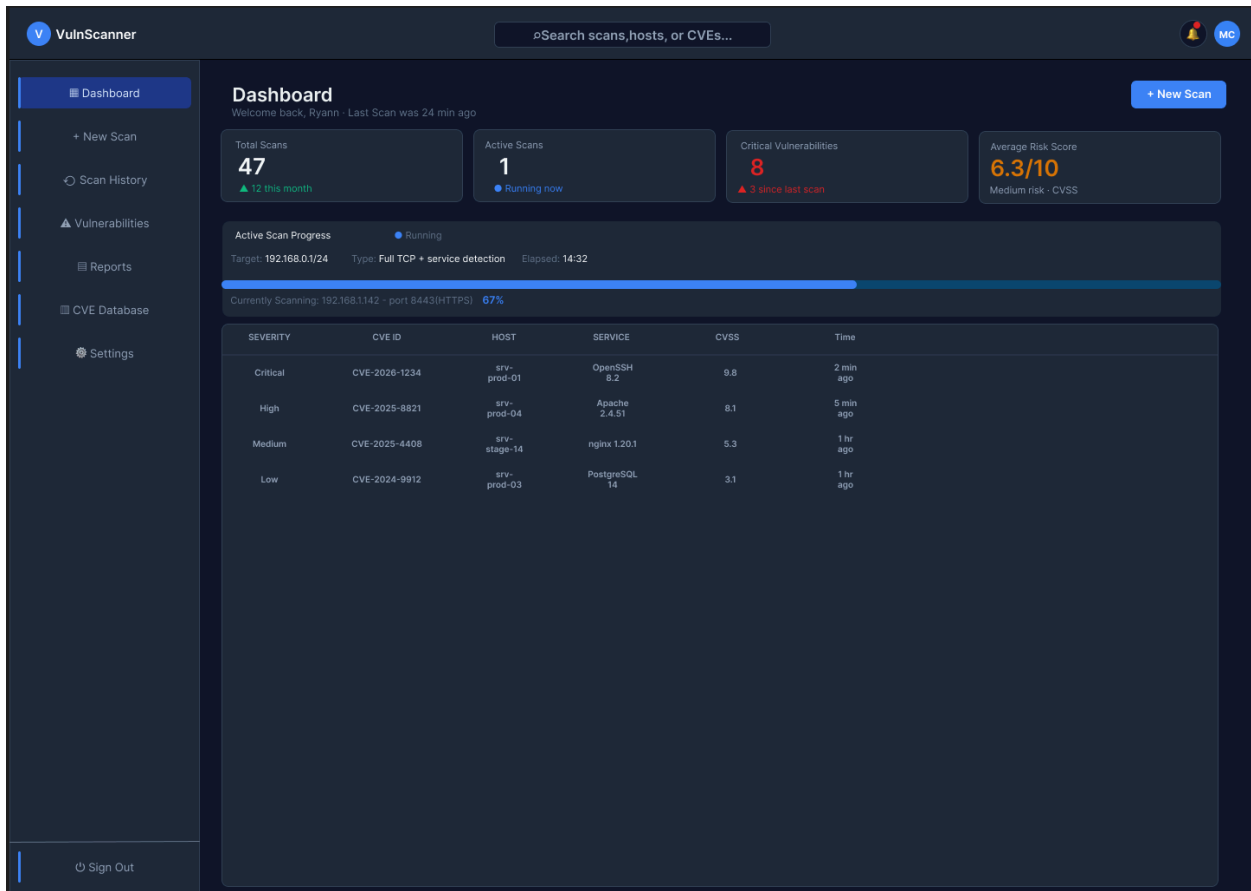


The screenshot above represents the scan results page. This page shows the list of vulnerabilities that have been found in the system and diagnosed. On the top portion of the screen we have multiple components describing the page. The header describes the current page we are viewing, underneath we have the description of the scan. Scan #048 describes the unique ID associated with the scan, 192.169.1/0/24 describes the IP subnet the scan is against. We also have the time the scan was completed in: 17:14, as well as the number of vulnerabilities found which is 24. This information alerts the user immediate awareness of the situation at hand before looking into any critical vulnerabilities. Underneath the heading we have four severity summary cards describing the type of severity (critical, high, medium, low) and how many have been found. These summary cards are provided with colors that contrast the background helping the analyst to easily identify the vulnerabilities that should be dealt with first. Critical has a high contrast red coloring bringing in alertness, high takes a red orange hue, with medium taking orange and low taking blue. Underneath these summary cards we have filtering chips. This allows the user to easily navigate through a wide range of vulnerabilities easily. This supports the analyzed results workflow feature found in Use Case 4. Underneath all of this we finally have the results table. This table details the vulnerability severity, CVE ID, the host, service, gives a CVSS score, and language description.

The screenshot above demonstrates the prototype's informational view working as intended. The summary cards explain to the user "how bad is this event?" before the user delves into their own

individual findings. The filter chips help narrow down on important vulnerabilities in the system, and the table preserves enough technicality for further analysis. There are several improvements that need to be made in the 491 portion of the course. The first would be to actually have a clickable row for each vulnerability. This would expand that vulnerability and give an in-depth analysis. Another improvement would be column sorting and persistent filters in the table. This allows for analysts to sort CVSS score, the affected host, services, and to save commonly used filter functions. Third would be an inline expandable row revealing a short summary without having to fully change pages allowing for quick reviews. Lastly, we want this view to integrate the notification system. This allows users to subscribe to certain alerts specific to hosts or severity thresholds directly from this screen tying in Use Cases 4 & 11 as well as our publish-subscribe design pattern.

Dashboard



The dashboard screenshot above presents a dark-themed interface displaying key user and scan information in a centralized layout. At the top of the dashboard, the user can view their username, recent activity, total number of scans completed on the account, currently active scans, and the total vulnerabilities identified along with an overall risk assessment rating. This provides users with an immediate overview of the current security posture of the scanned systems.

The middle section of the dashboard provides more detailed live scan information, including the target IP address, the type of scan currently running, and the elapsed scan time. Since only one

scan is active in the screenshot, a single monitoring module is displayed. This section is designed to give users real-time visibility into scan progress without overwhelming the interface with unnecessary detail.

The lower section of the dashboard presents the most detailed vulnerability information. This includes the CVE ID, severity level, affected host, detected service, CVSS score, and the scan completion timestamp. By displaying technical vulnerability data in an organized format, users can quickly identify and prioritize the most critical findings.

On the left side of the interface, a navigation sidebar allows users to move between the major modules of the application. These modules include initiating a new scan, viewing scan history, accessing detailed vulnerability information, generating reports, browsing the CVE database, and adjusting application settings. Some of these navigation options are also accessible through interactive widgets located on the right side of the dashboard, providing users with multiple pathways to frequently used features and improving overall usability.

The layout of the dashboard reflects the project's overall UX philosophy of balancing technical depth with usability. Information is separated into clear visual sections so users can quickly distinguish between scan activity, vulnerability findings, and navigation controls without excessive cognitive load. The interface also prioritizes visibility of system status through persistent scan monitoring and real-time updates, ensuring users always understand what the application is currently doing and what actions are available to them.

New Scan

The screenshot shows the 'New Scan' interface of the VulnScanner application. The interface has a dark theme. On the left is a sidebar with navigation links: Dashboard, + New Scan (highlighted), Scan History, Vulnerabilities, Reports, CVE Database, and Settings. At the bottom of the sidebar is a 'Sign Out' link. The main content area is titled 'New Scan' with a subtitle 'Configure scan parameters and target'. It contains several input fields and a list of scan types. The 'Target IP / hostname / CIDR' field has the value '192.168.1.0/24'. The 'Port Range' field has the value '1 - 65535'. The 'Scan Intensity' dropdown is set to 'Normal (T3)'. The 'CVE database' dropdown is set to 'NVD + NIST (latest)'. The 'Scan type' section has three options: 'Full TCP + Service Detection' (selected with a radio button), 'Quick TCP Scan', and 'Stealth Scan'. Each option has a brief description. At the bottom of the form are 'Start Scan' and 'Cancel' buttons. A search bar is located at the top right of the main area, and a user profile icon is in the top right corner.

VulnScanner

Search scans, hosts, or CVEs...

Dashboard

+ New Scan

Scan History

Vulnerabilities

Reports

CVE Database

Settings

Sign Out

New Scan

Configure scan parameters and target

Target IP / hostname / CIDR: 192.168.1.0/24

Port Range: 1 - 65535

Scan Intensity: Normal (T3)

CVE database: NVD + NIST (latest)

Scan type:

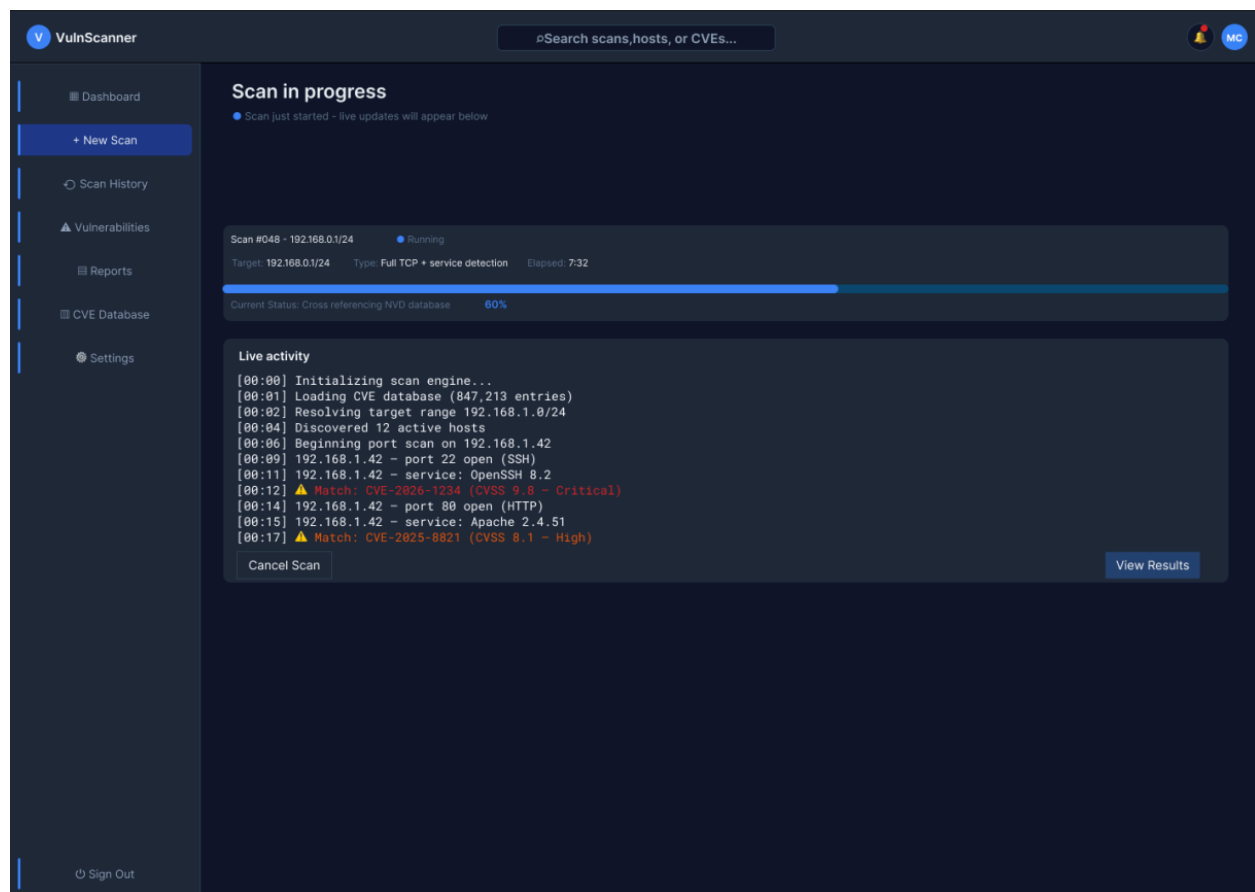
- ☒ Full TCP + Service Detection
Comprehensive scan with version detection. Recommended
- ☐ Quick TCP Scan
Top 1000 common ports only. Faster but less thorough
- ☐ Stealth Scan
SYN scan with reduced footprint. Requires elevated privileges

Start Scan Cancel

This is the New Scan screen of the Vulnerability Scanner. This is where the user inputs information for the scan before running it. The user can enter the IP address, hostname, or CIDR range they want to scan. The port section they insert which ports they would like to scan. In the scan intensity section they choose how in depth the scan will be. The CVE database section allows the user to pick which database they would like to use for the scan. Lastly, the scan type is chosen out of three options. Out of the three options, Full TCP plus Service Detection described as a comprehensive scan with version detection and marked as recommended, Quick TCP Scan described as covering the top 1000 common ports only for faster, but less thorough results, and Stealth Scan described as a SYN scan with reduced footprint requiring elevated privileges.

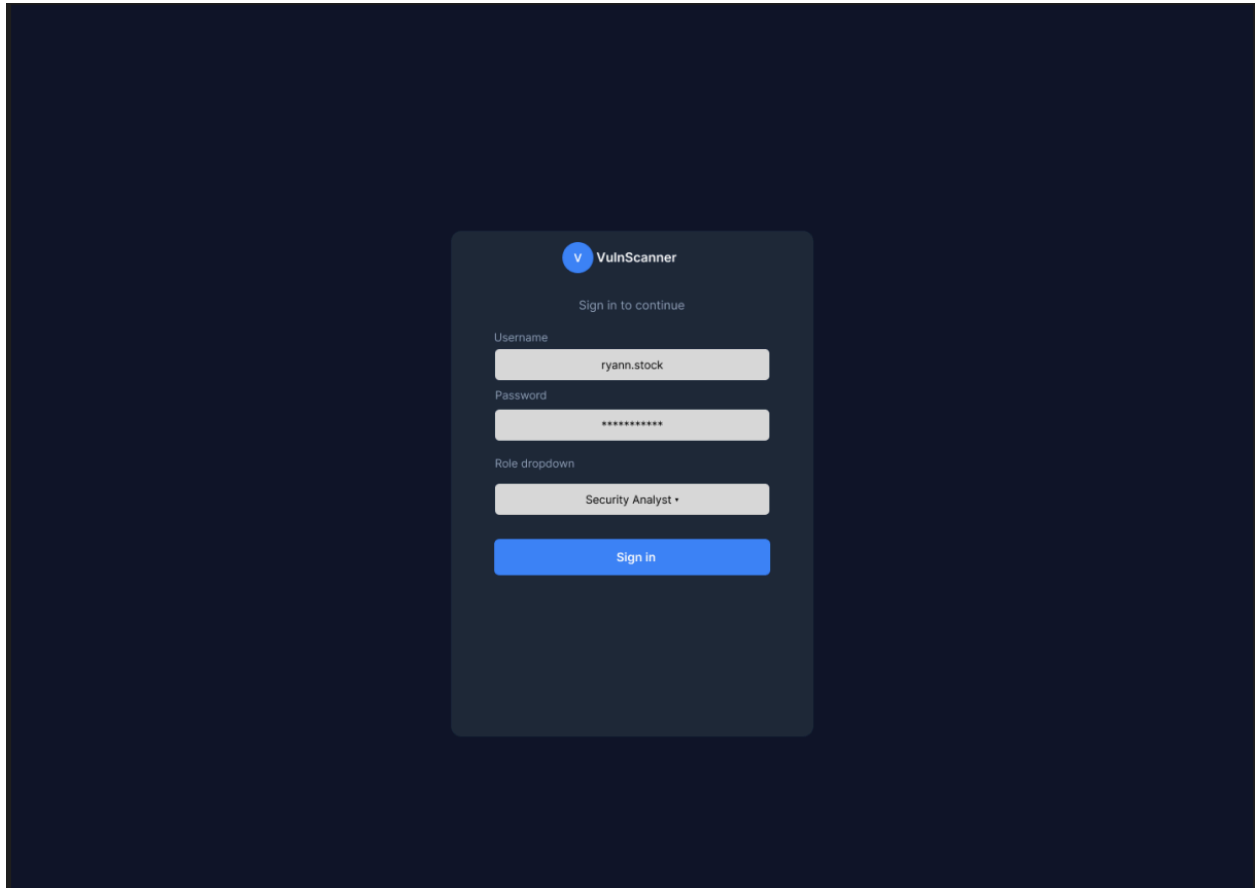
A Start Scan button and a Cancel button complete the screen. This screen directly maps to Use Case 1, which covers running the vulnerability scanner, and to Use Case 3, which covers configuring scan targets and parameters. The scan type options reflect the three scan modes identified in the UX Design mockup and give users of varying technical backgrounds an appropriate entry point into the scanning workflow. A less experienced user can select the recommended Full TCP option without having to understand the technicalities between the different scan types. A more user such a security analyst would choose a stealth scan option to use in a more sensitive environment. After this page the user would be directed towards the scan in progress page.

Scan in Progress



After the user clicks Start Scan the prototype transitions to the scan in progress screen. This Screen confirms that the prototype responds to user input and moves between states rather than remaining static. The screen header displays the scan identifier as Scan 048 against the target subnet 192.168.0.1/24 with a running status indicator, the selected scan type and elapsed time. A progress bar below shows the current completion percentage alongside a current status label reading cross referencing the NVD database at 60 percent. The lower portion of the screen displays a live activity log in a terminal style text panel. The log shows time stamped entries beginning with scan engine initialization, CVE database loading with 847,213 entries, target range resolution, and host discovery showing 12 active hosts found. As the log progresses it shows port discovery on a specific host, service identification as OpenSSH 8.2, and two highlighted vulnerability matches shown in red and orange text respectively, CVE-2026-1234 rated Critical and CVE-2025-8821 rated High. This live log display reflects the publish-subscribe pattern described in the Architecture section, where the scan engine publishes events and the dashboard subscribes and displays them in real time. A Cancel Scan button and a View Results button are available at the bottom of the screen. This screen supports Use Case 8, which covers real-time scan progress monitoring.

Log in UI



The screen presents the VulnScanner logo and brand name at the top, followed by a username field, a password field, and a role dropdown currently set to Security Analyst. A Sign In button completes the form. The role dropdown is a notable design decision because the vulnerability scanner is intended to support users with different permission levels. A Security Analyst role would have full access to scan configuration and results, while a lower privilege role might have read-only access to reports. After the user enters sample credentials and clicks Sign In, the prototype transitions to the dashboard, demonstrating that the application responds to user interaction rather than displaying static screens. This screen directly supports Use Case 7, which covers user authentication and access control.

Scan History

SCAN ID	TARGET	TYPE	STATUS	STARTED	DURATION	FINDINGS
#048	192.168.1.0/24	Full TCP	Running 67%	Today, 2:14 PM	14m 32s	3 7 9
#047	10.0.0.0/16	Quick TCP	Completed	Today, 11:02 AM	8m 17s	2 5
#046	srv-prod-01.local	Full TCP	Completed	Yesterday, 6:45 PM	3m 41s	1 2
#045	192.168.1.42	Stealth	Completed	Yesterday, 2:18 PM	5m 03s	3 1
#044	172.16.0.0/24	Full TCP	Completed	2 days ago, 9:30 AM	22m 11s	2 4 12
#043	192.168.99.0/24	Quick TCP	Failed	3 days ago, 4:12 PM	0m 14s	—
#042	10.0.5.0/24	Full TCP	Completed	4 days ago, 10:08 AM	18m 52s	3 6 4
#041	srv-stage-08.local	Full TCP	Completed	5 days ago, 3:22 PM	2m 49s	2

Our scan history is an important feature because it pulls records of past scans for user purposes like record keeping and analysis of previous scans. Every scan conducted should call the system to record the information and then add it to an accessible database for the users. This database will be optimized and streamlined for quick reads in the backend for returning information to the frontend and tuned with menu options for easy queries on the client side interface. This will be a component of the program where a huge focus will be on applying design patterns and philosophies. Scans are given ID numbers to help keep track, the target should tell us where the scans were conducted, status tells us whether our program completed its scanner routine, and gives us a time to record. The most important column however is the findings, that indicates immediately to users of how many problems and the severity for each number i.e red is the most severe. The scan history includes further details pertaining to each scan. Each scan in the history includes a brief summary of the scan so it is easily identifiable. Scan history also shows whether the scan was completed, in progress, or failed. The top of the section gives the user the opportunity to filter by the types of scans depending on the state of

the scan. At the upper right corner, the user is able to conduct a new scan after viewing the scan history.

Reports

The screenshot shows the VulnScan web interface. On the left is a sidebar with navigation links: Dashboard, New Scan, Scan History, Vulnerabilities, Reports (selected), CVE Database, Settings, and Sign out. The main content area is titled 'Vulnerability Report' and includes a breadcrumb 'Back to results > CVE-2026-1234 > srv-prod-01'. At the top right of the main area are buttons for 'Export CSV' and 'Export PDF'. The report is divided into three main sections:

- Primary Information:** Displays a 'CRITICAL' severity badge, a large red '9.8' CVSS v3.1 Base Score, and the title 'CVE-2026-1234 — OpenSSH Remote Code Execution'. A plain language description follows: 'A specially crafted SSH authentication payload can trigger a use-after-free condition in OpenSSH versions 8.0 through 8.2, allowing remote attackers to execute arbitrary code with the privileges of the SSH daemon. Exploitation requires no authentication and can be performed over the network.'
- Affected systems:** A table listing the affected hosts, IP addresses, services, and versions.
- Recommended remediation:** A list of five steps to address the vulnerability.
- Vulnerability details:** A table providing metadata about the CVE, including its ID, publication date, CVSS score, attack vector, complexity, privileges, user interaction, confidentiality, integrity, and availability.
- References:** A list of external links for more information, including NVD, MITRE, and OpenSSH security advisories.

HOST	IP ADDRESS	SERVICE	VERSION
srv-prod-01	192.168.1.42	SSH	OpenSSH 8.2
srv-prod-05	192.168.1.51	SSH	OpenSSH 8.1

Field	Value
CVE ID	CVE-2026-1234
Published	Jan 18, 2026
CVSS v3.1	9.8 / 10
Attack Vector	Network
Complexity	Low
Privileges	None required
User interaction	None required
Confidentiality	High impact
Integrity	High impact
Availability	High impact

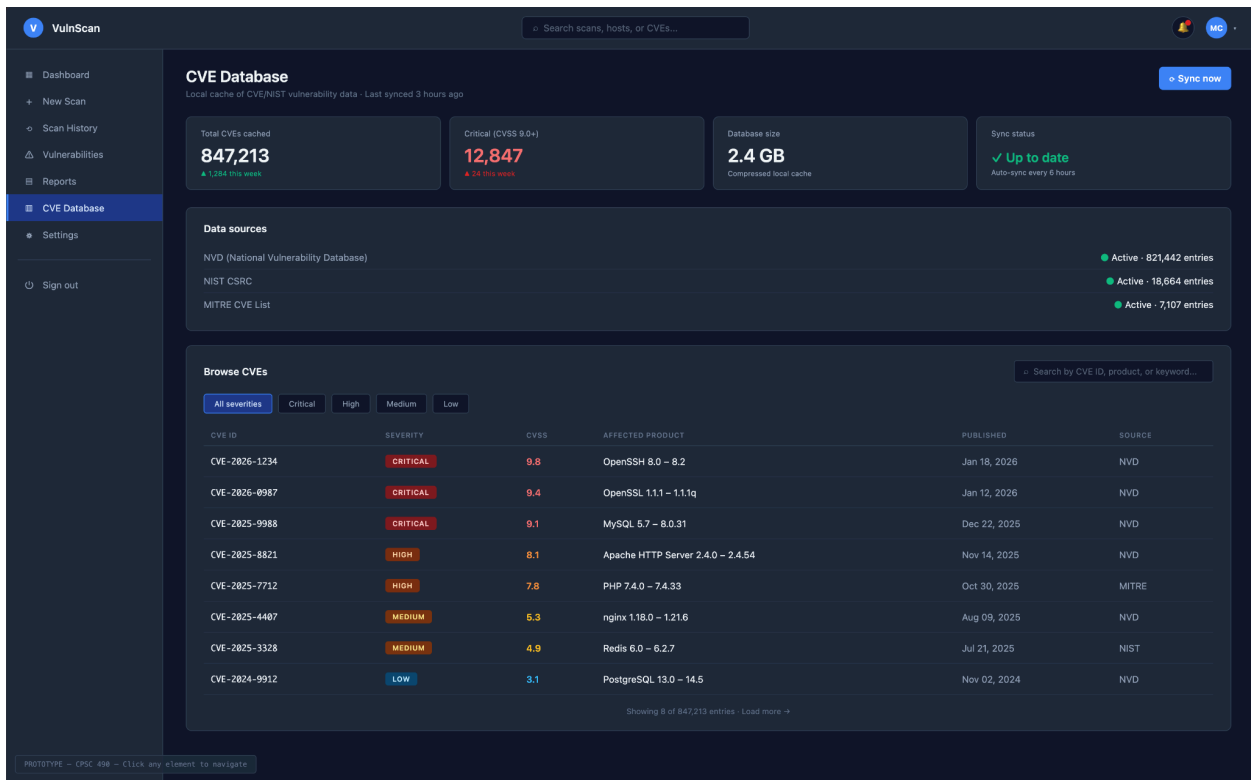
1. Upgrade OpenSSH to version 8.3 or later on all affected hosts
2. Restrict SSH access to known IP ranges via firewall rules
3. Enable SSH key-based authentication and disable password login
4. Monitor SSH logs for unusual authentication attempts
5. Re-scan affected hosts after patching to confirm remediation

The vulnerability report screen provides the user with a detailed breakdown of a single CVE finding pulled from the scan results. The screen is accessible by selecting any vulnerability entry from the results table, and a back to results breadcrumb at the top allows the user to return to the full results list without losing their place. The report is divided into three main sections across a two-column layout.

The left column contains the primary vulnerability information. At the top a Critical severity badge is displayed alongside the CVSS v3.1 base score of 9.8 rendered in a large red font, immediately communicating the urgency of the finding before the user reads any further detail. Below this the CVE identifier and vulnerability name are displayed as a header, in this case CVE-2026-1234 describing an OpenSSH Remote Code Execution vulnerability. A plain language description follows, explaining that a specially crafted SSH authentication payload can trigger a use-after-free condition in OpenSSH versions 8.0 through 8.2, allowing remote attackers to execute arbitrary code with the privileges of the SSH daemon without requiring authentication. This plain language description is a deliberate UX decision that makes the report accessible to non-specialist users such as small business owners or junior administrators who may not be familiar with technical vulnerability nomenclature.

Below the description an affected systems panel lists every host in the scanned network where this vulnerability was detected. Each row shows the hostname, IP address, detected service, and specific version. In this example two hosts are affected, srv-prod-01 at 192.168.1.42 running OpenSSH 8.2 and srv-prod-05 at 192.168.1.51 running OpenSSH 8.1. Listing all affected hosts in one view rather than requiring the user to cross-reference the results table separately reduces the number of steps needed to assess the full scope of a single vulnerability across the network.

CVE Database



The CVE database screen provides the user with direct access to the local cache of vulnerability data that the scanner uses during scan operations. Rather than requiring the user to visit external sources such as the NIST National Vulnerability Database or the MITRE CVE list separately, the application maintains a locally synchronized copy of these databases and presents them in a searchable, filterable interface within the application itself. This design decision reflects the CVECacheRepository component described in the Architecture and Design section, where caching vulnerability data locally reduces dependency on external API availability during active scans and improves scan speed by eliminating repeated network requests for the same vulnerability records.

At the top of the screen four summary cards provide an at-a-glance status of the local database. The first card shows the total number of CVEs currently cached at 847,213 entries with a note

that 1,284 new entries were added this week. The second card highlights the number of critical severity CVEs with a CVSS score of 9.0 or higher, currently 12,847, with 24 new critical entries added this week shown in red to draw immediate attention. The third card displays the total database size as 2.4 gigabytes stored as a compressed local cache. The fourth card shows the sync status as up to date with a green checkmark and a note that the database auto-syncs every 6 hours. A Sync Now button in the top right corner allows the user to manually trigger an update outside the automatic schedule, which is useful when a high-profile vulnerability is publicly disclosed and the user needs the latest data before the next scheduled sync. Together these cards give the user confidence that the scanner is working from current and complete vulnerability information, which is critical for the accuracy of scan results. This directly supports Use Case 5, which covers pulling CVE and NIST data via API and updating the local vulnerability database.

Below the summary cards a data sources panel lists the three active sources feeding the local cache. The NVD National Vulnerability Database contributes 821,442 entries, NIST CSRC contributes 18,664 entries, and the MITRE CVE List contributes 7,107 entries. Each source shows a green active indicator confirming the connection is healthy and data is being received. Displaying the individual source contribution counts gives technically sophisticated users transparency into where the scanner's vulnerability intelligence comes from, which is important for users like Alex the IT administrator who may need to justify the scanner's data sources to a compliance team or management. The lower portion of the screen presents a browsable table of all cached CVEs. Severity filter chips at the top allow the user to narrow the table to All Severities, Critical, High, Medium, or Low findings.

References

Cooper, A., Reimann, R., Cronin, D., & Noessel, C. (2014). *About face: The essentials of interaction design* (4th ed.). Wiley.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley Professional.

Dubberly, H. (2001, March 1). *Alan Cooper and the goal-directed design process*. Dubberly Design Office.

<https://www.dubberly.com/articles/alan-cooper-and-the-goal-directed-design-process.html>

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Figma. (n.d.). *What is information architecture?*. Figma Resource Library.

<https://www.figma.com/resource-library/what-is-information-architecture/>

National Institute of Standards and Technology. (2024). *National vulnerability database*. U.S. Department of Commerce. <https://nvd.nist.gov>

MITRE Corporation. (2024). *Common vulnerabilities and exposures (CVE)*. <https://cve.mitre.org>

Rodgers, J. (2025, April 09). *What is hub and spoke model*. Netcom Learning.

<https://www.netcomlearning.com/blog/what-is-hub-and-spoke-model>

Hasan, S. (2019, May 26). *Pipe and filter architecture*. Medium.

<https://syedhasan010.medium.com/pipe-and-filter-architecture-bd7babdb908>

Microsoft. (n.d.). *Pipes and filters pattern*. Microsoft Azure Architecture Center.

<https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters#context-and-problem>

Mieng, L (2021, January 4.). *What is vulnerability scanning and how does it work?*. Fortra.

<https://www.fortra.com/blog/what-is-vulnerability-scanning-and-how-does-it-work>

Vicarius. (2025, November 24). *The complete guide to vulnerability scanning (2025 edition)*. Vicarius.

<https://www.vicarius.io/articles/the-complete-guide-to-vulnerability-scanning-2025-edition>

Shvets, A. (n.d.). *Strategy*. Refactoring.Guru. <https://refactoring.guru/design-patterns/strategy>

Siddiqui, L. (n.d.). *Vulnerability scanning: A complete guide*. Splunk.

https://www.splunk.com/en_us/blog/learn/vulnerability-scanning.html

GeeksforGeeks. (2026, May 7). *Strategy pattern – Set 1*. GeeksforGeeks.

<https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/>

OWASP Foundation. (2024). *OWASP Web Security Testing Guide*.

[\[https://owasp.org/www-project-web-security-testing-guide\]](https://owasp.org/www-project-web-security-testing-guide)<https://owasp.org/www-project-web-security-testing-guide>

Rohrer, C. (2022, July 17). *When to Use Which User-Experience Research Methods*. Nielsen Norman Group.

<https://www.nngroup.com/articles/which-ux-research-methods/>

ISO/IEC. (2023). ISO/IEC 25010: Systems and software engineering - System and software quality models. International Organization for Standardization.
[\[https://www.iso.org/standard/35733.html\]](https://www.iso.org/standard/35733.html)(<https://www.iso.org/standard/35733.html>)

Repository (design pattern). (2026, February 15). *Wikipedia*. Retrieved May 12, 2026, from [https://simple.wikipedia.org/w/index.php?title=Repository_\(design_pattern\)&oldid=10750997](https://simple.wikipedia.org/w/index.php?title=Repository_(design_pattern)&oldid=10750997).

Wikipedia contributors. (2026, January 27). Command Query Responsibility Segregation. In *Wikipedia, The Free Encyclopedia*. Retrieved 21:26, May 12, 2026, from https://en.wikipedia.org/w/index.php?title=Command_Query_Responsibility_Segregation&oldid=1335191085

What is UX Research? (2016, June 2). IxDF - Interaction Design Foundation.
<https://ixdf.org/literature/topics/ux-research>

Forum of Incident Response and Security Teams. (2023). *Common Vulnerability Scoring System version 4.0 specification document*. <https://www.first.org/cvss/specification-document>

Figma. (n.d.). *Guide to prototyping in Figma*.
<https://help.figma.com/hc/en-us/articles/360040314193-Guide-to-prototyping-in-Figma>

National Institute of Standards and Technology. (n.d.). *National Vulnerability Database*. U.S. Department of Commerce. <https://nvd.nist.gov/>

Nielsen Norman Group. (2024). *10 usability heuristics for user interface design*.
<https://www.nngroup.com/articles/ten-usability-heuristics/>

Nielsen Norman Group. (2025). *Personas make users memorable*.
<https://www.nngroup.com/articles/persona/>

OWASP Foundation. (n.d.). *Web Security Testing Guide*.
<https://owasp.org/www-project-web-security-testing-guide/>

Cappiello, A., Mocenni, C., & Madeo, D. (2015). A secure, scalable and versatile multi-layer client-server architecture for remote intelligent data processing. *Journaling of Reliable Intelligent Environments*, 1(2-4), 173-187. <https://doi.org/10.1007/s40860-015-0007-1>

Elsevier. (n.d.-a). *Client-server architecture*. ScienceDirect Topics. Retrieved May 7, 2026, from <https://www.sciencedirect.com/topics/computer-science/client-server-architecture>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patters: Elements of reusable object-oriented software*. Addison-Wesley.

Joshi, C., & Singh, U.K. (2018). Layered architecture for assessment of security vulnerabilities. *International Journal of Creative Research Thoughts*, 6(2), 1448-1456.
<https://ijcrt.org/papers/IJCRT1812616.pdf>